Mohamed bin Zayed University of Artificial Intelligence

Lecture 7

Architecture design 2

Fabio Pizzati

AI1220: Software, web, mobile engineering

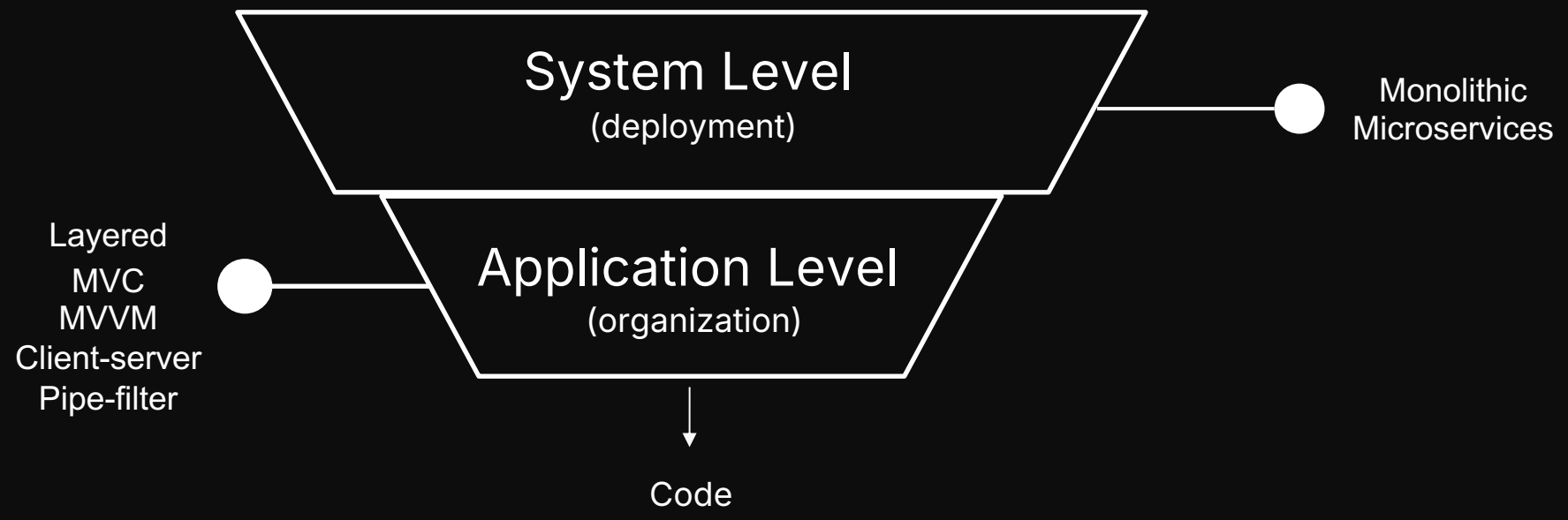
A small recap

- We need design
- We have different level of design
- We have good design principles

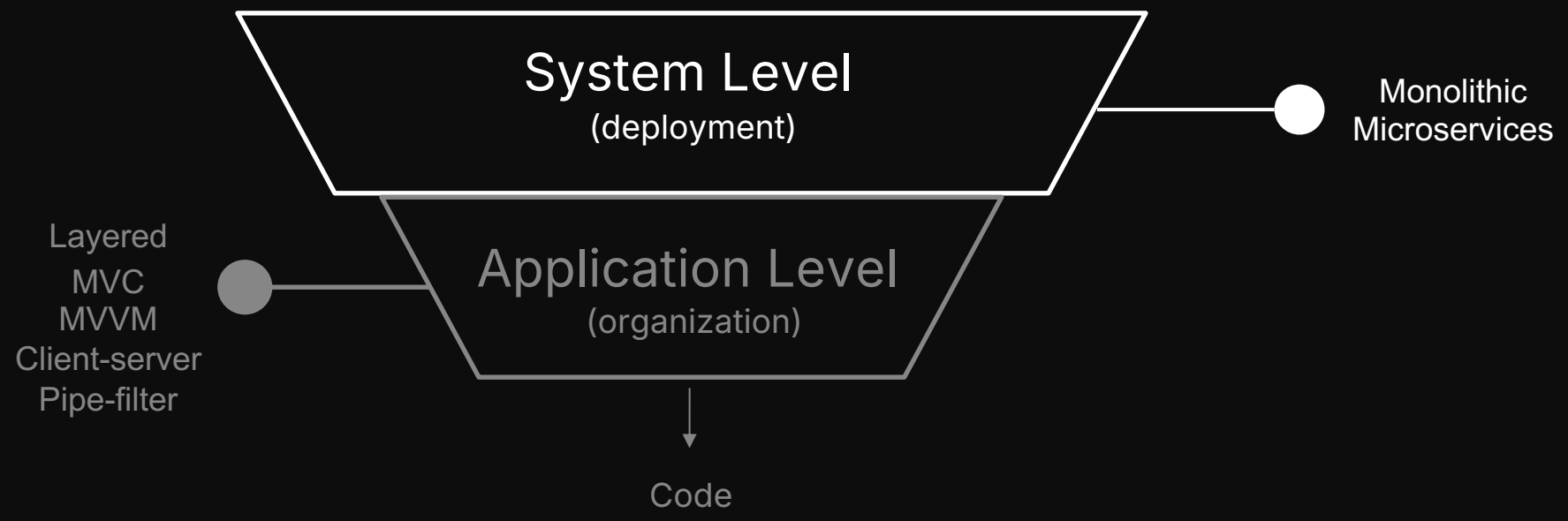
But in practice?

Let’s not reinvent the wheel

Design patterns



Design patterns



System level patterns

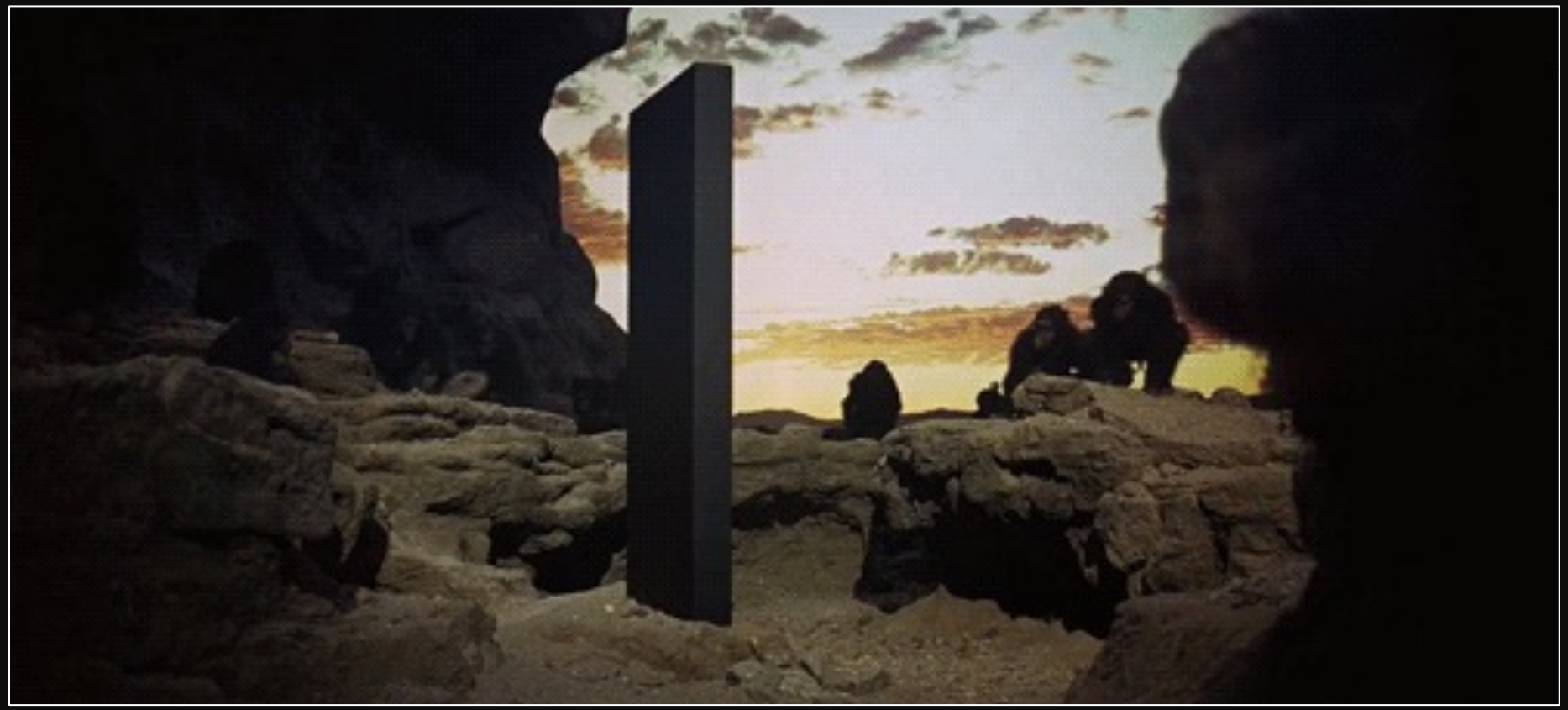
Monolithic

All code runs together

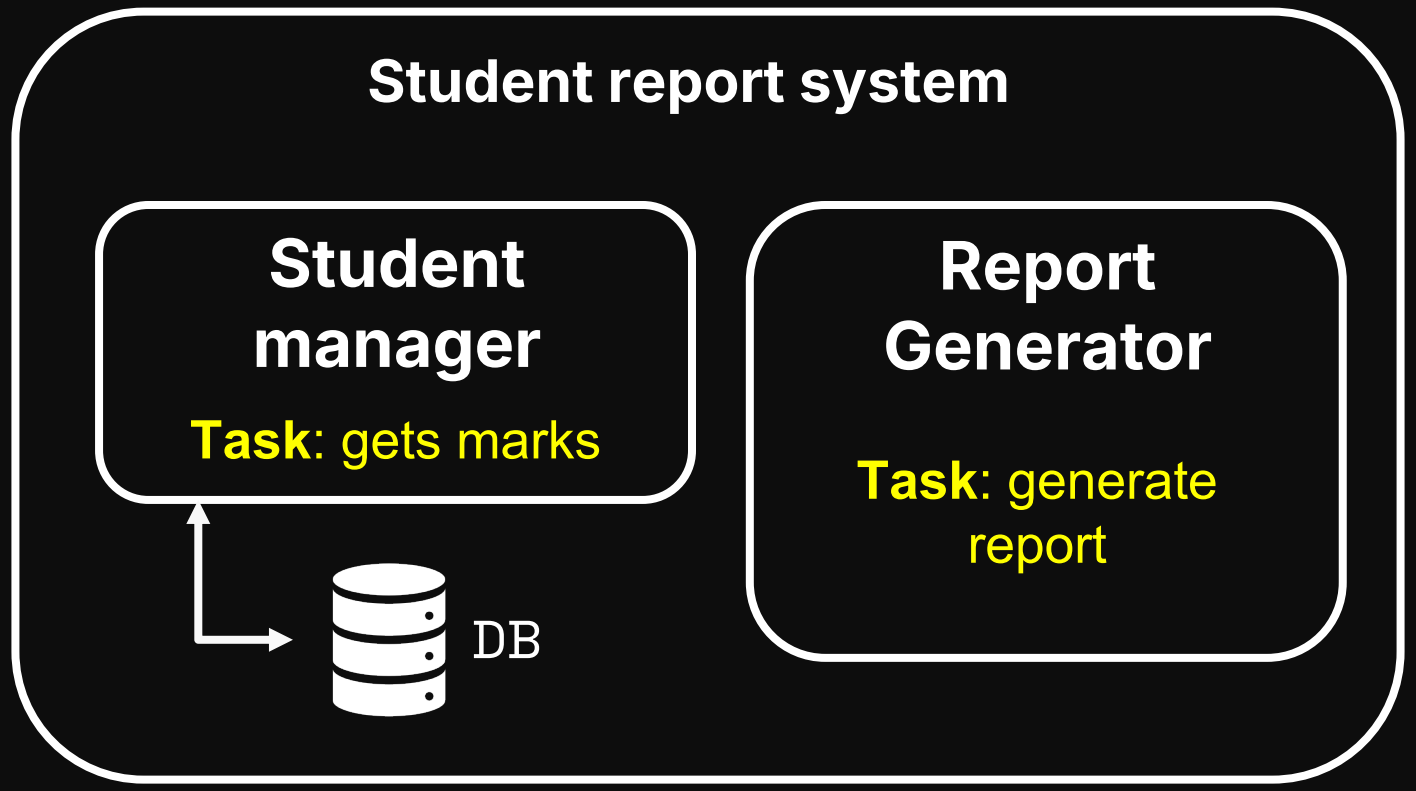
Microservices

Independent programs

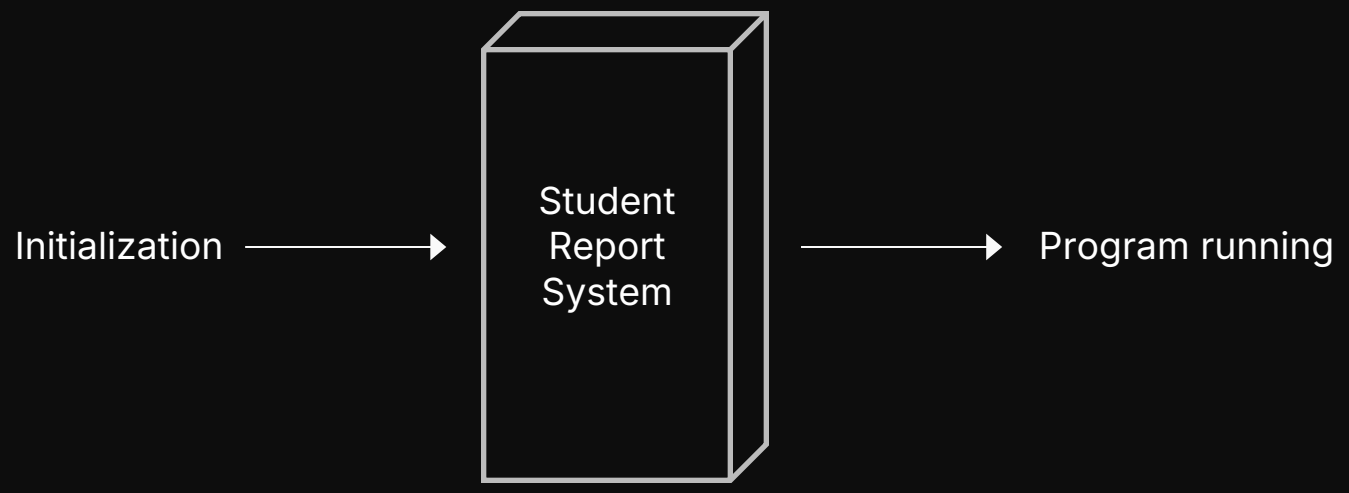
2001: a Space Odyssey



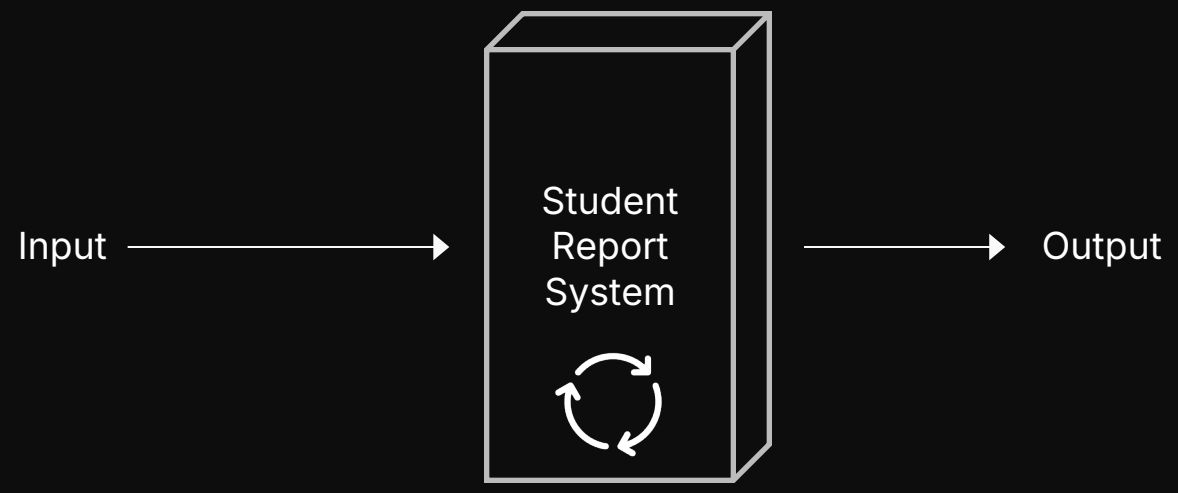
Recall


Student report system
Student manager
Task: gets marks
Report Generator
Task: generate report
DB

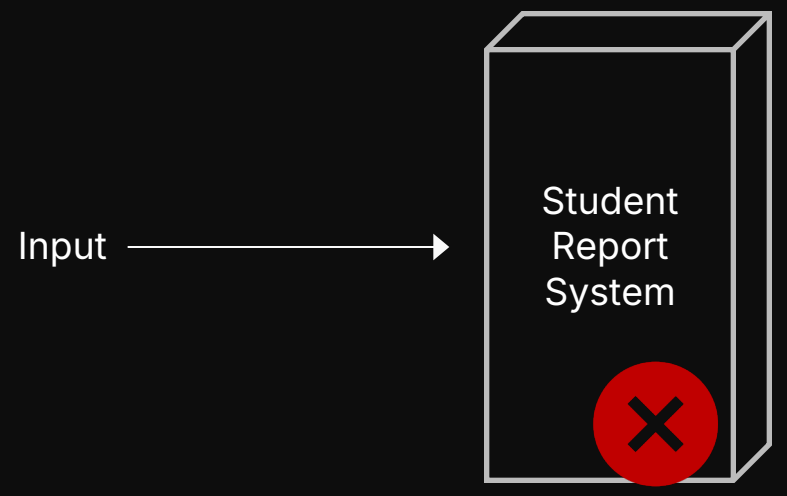
Monolith



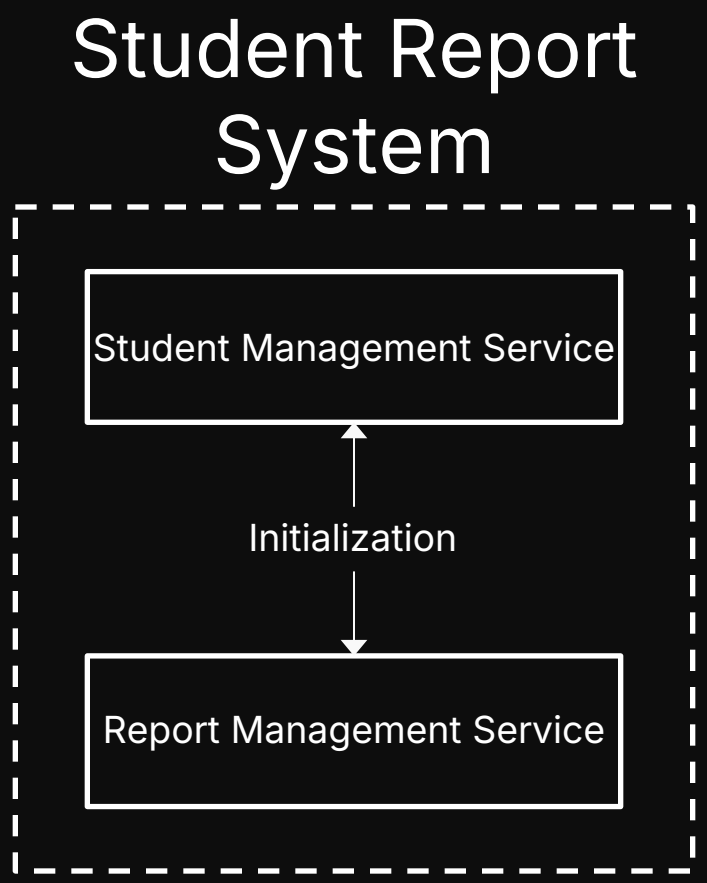
Monolith



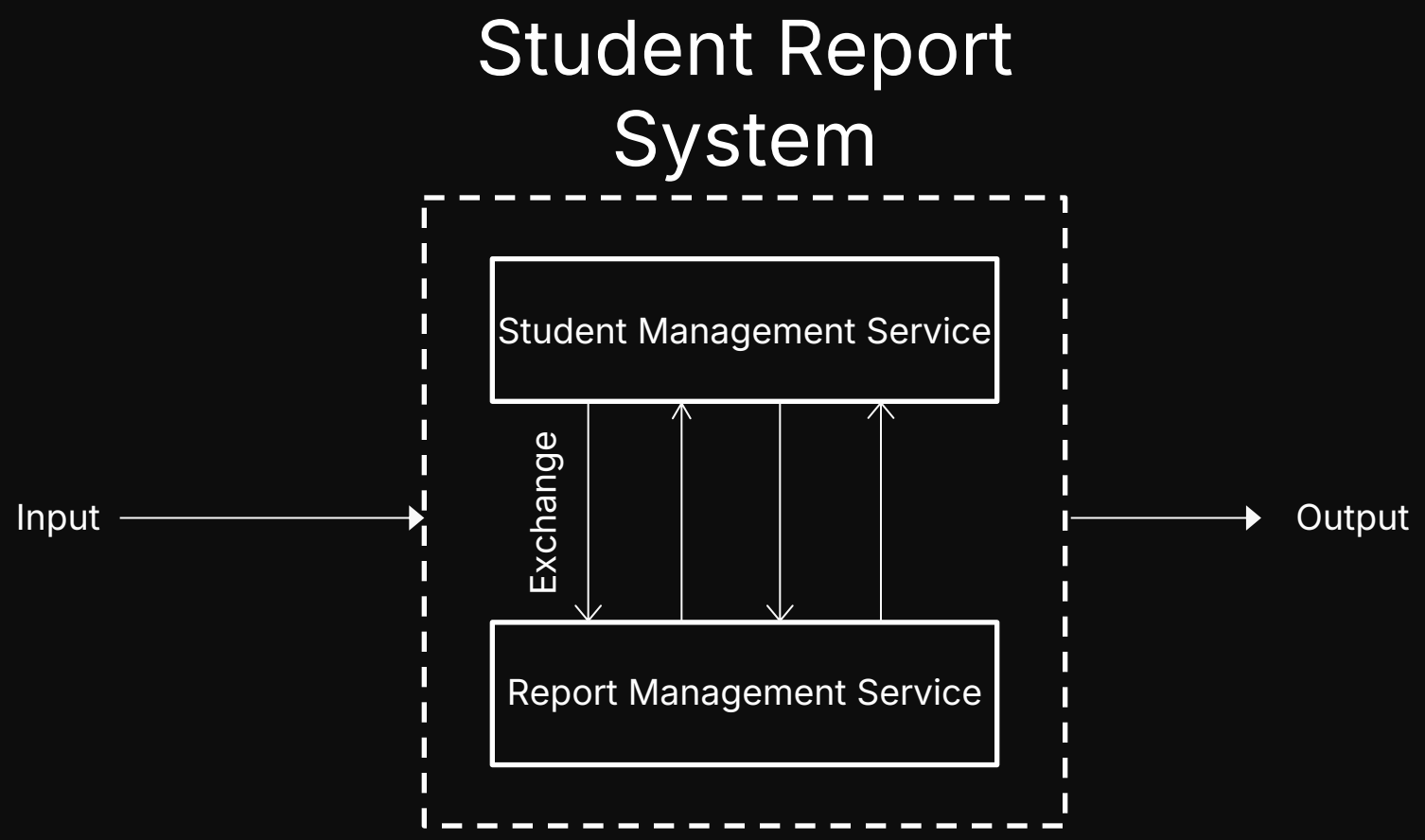
More in details



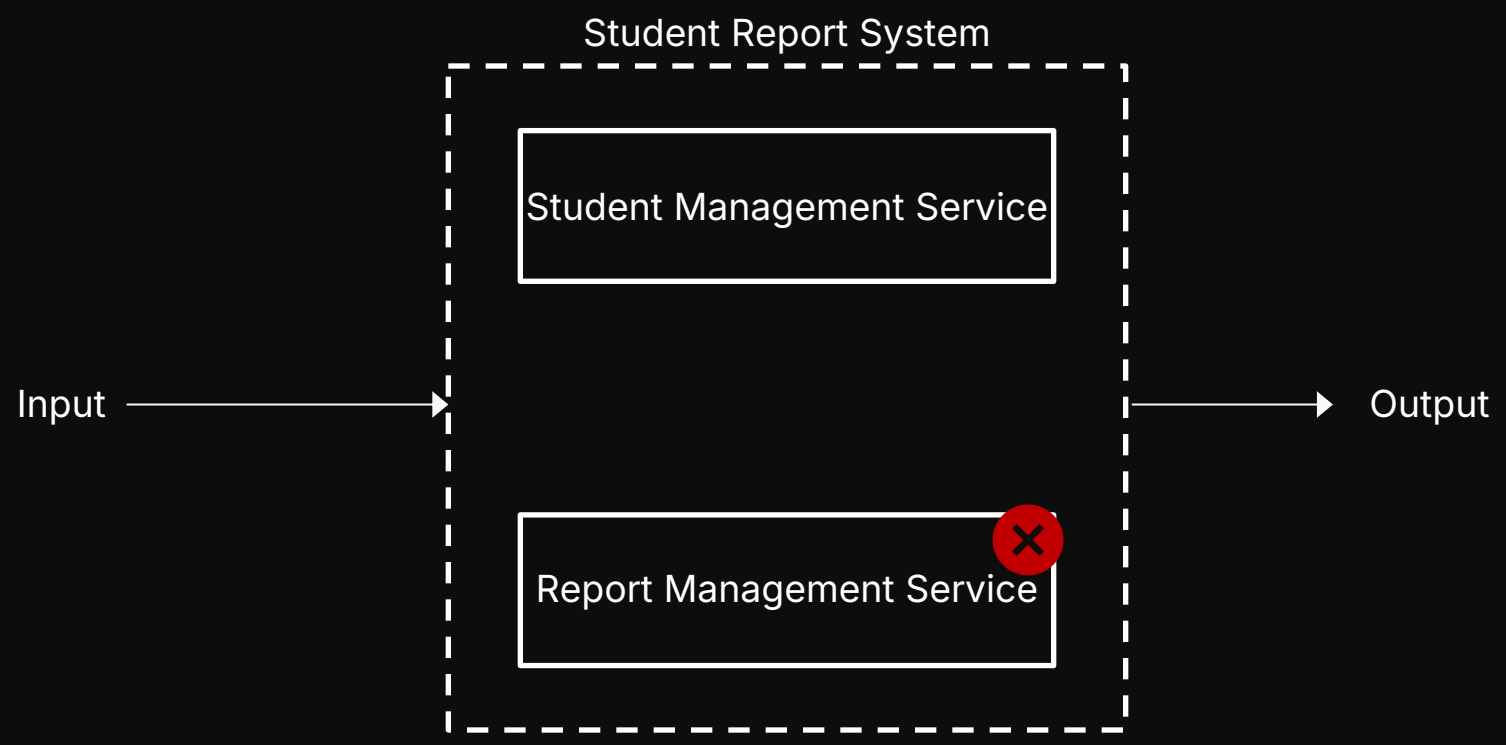
Microservices



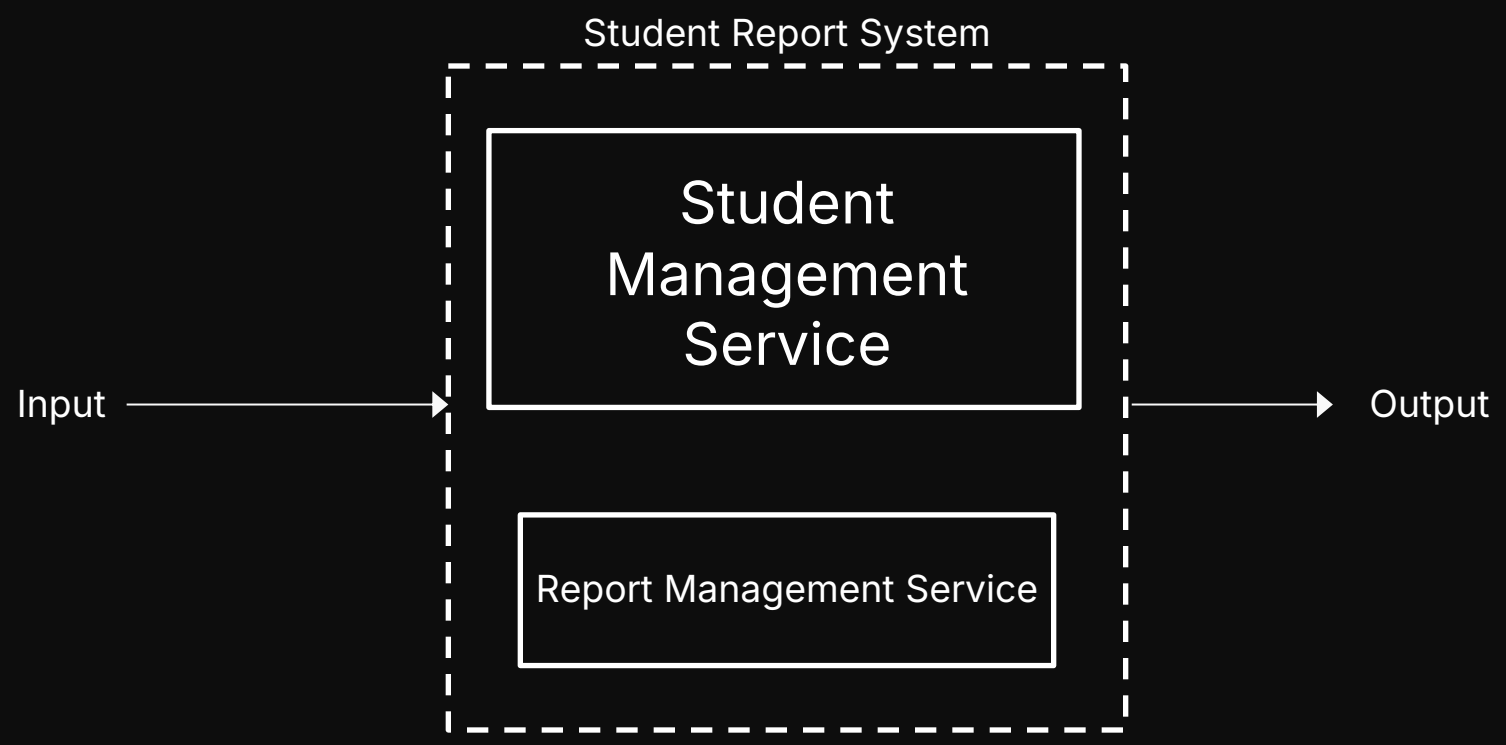
Microservices



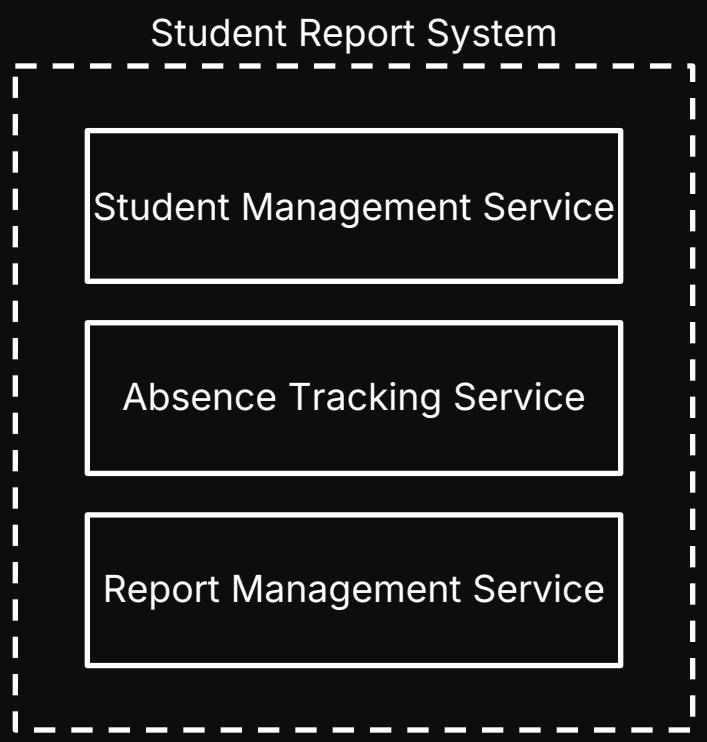
Failures


Student Report System
Student Management Service
Input
Output
Report Management Service

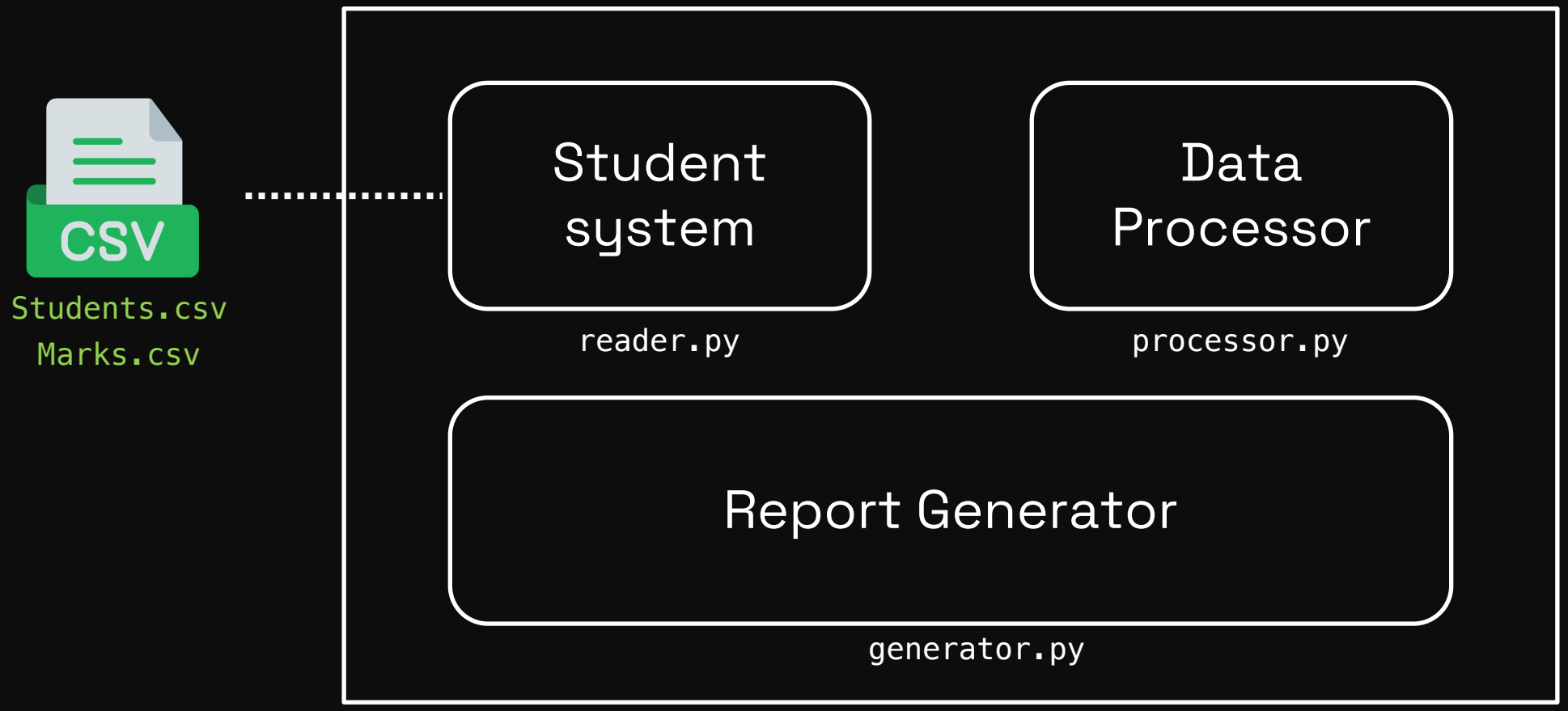
Scaling

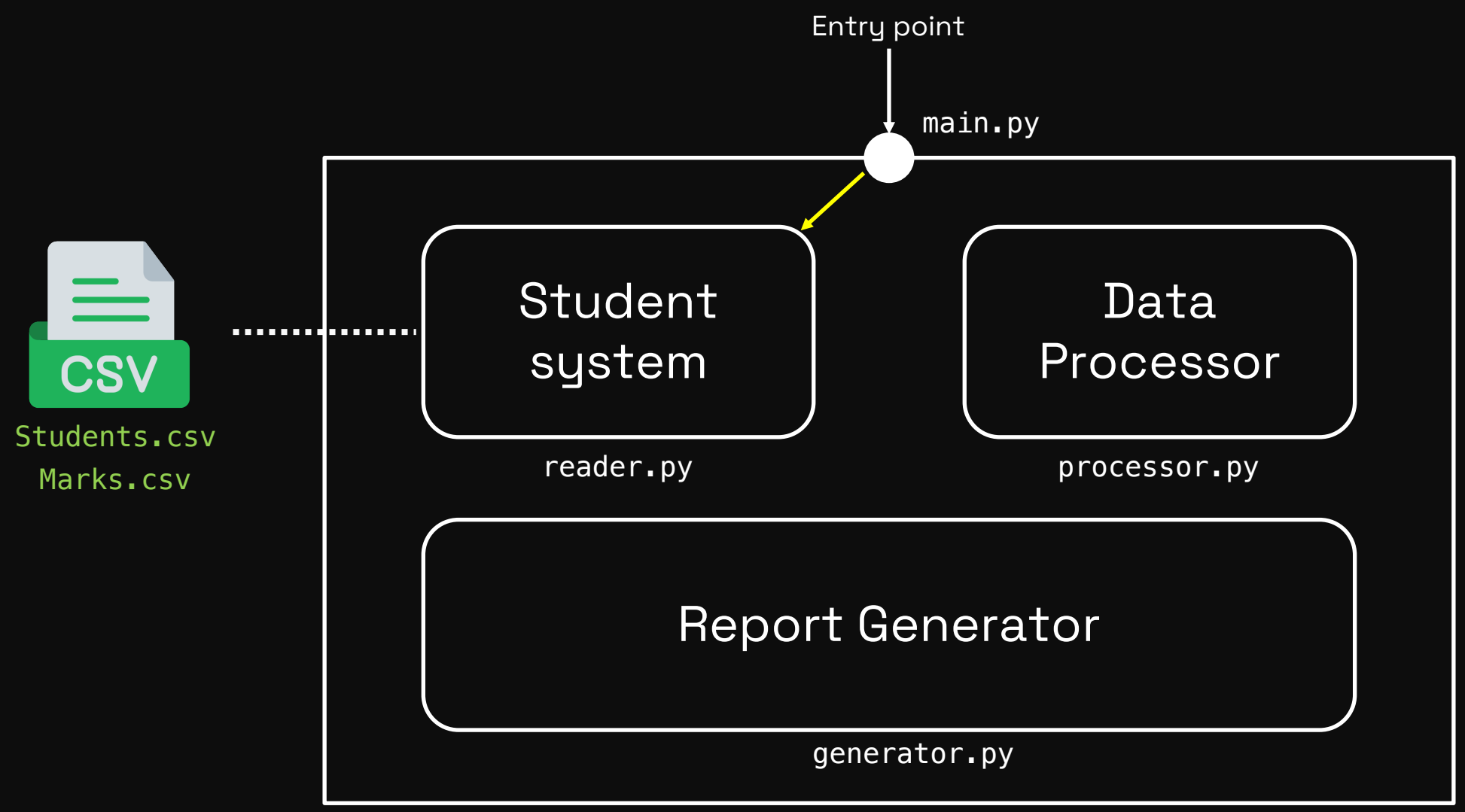

Student Report System
Student Management Service
Input
Output
Report Management Service

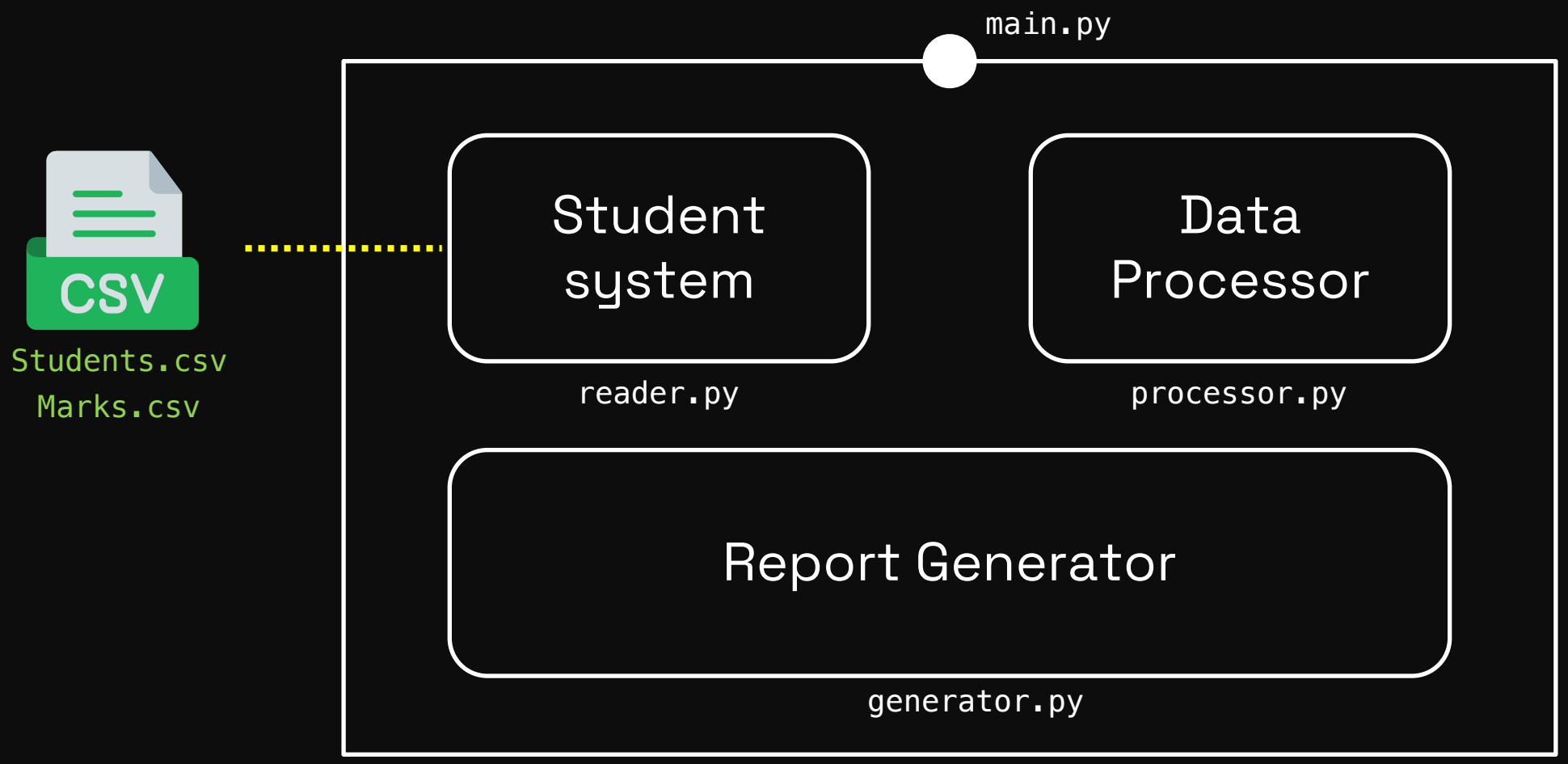
Extensions

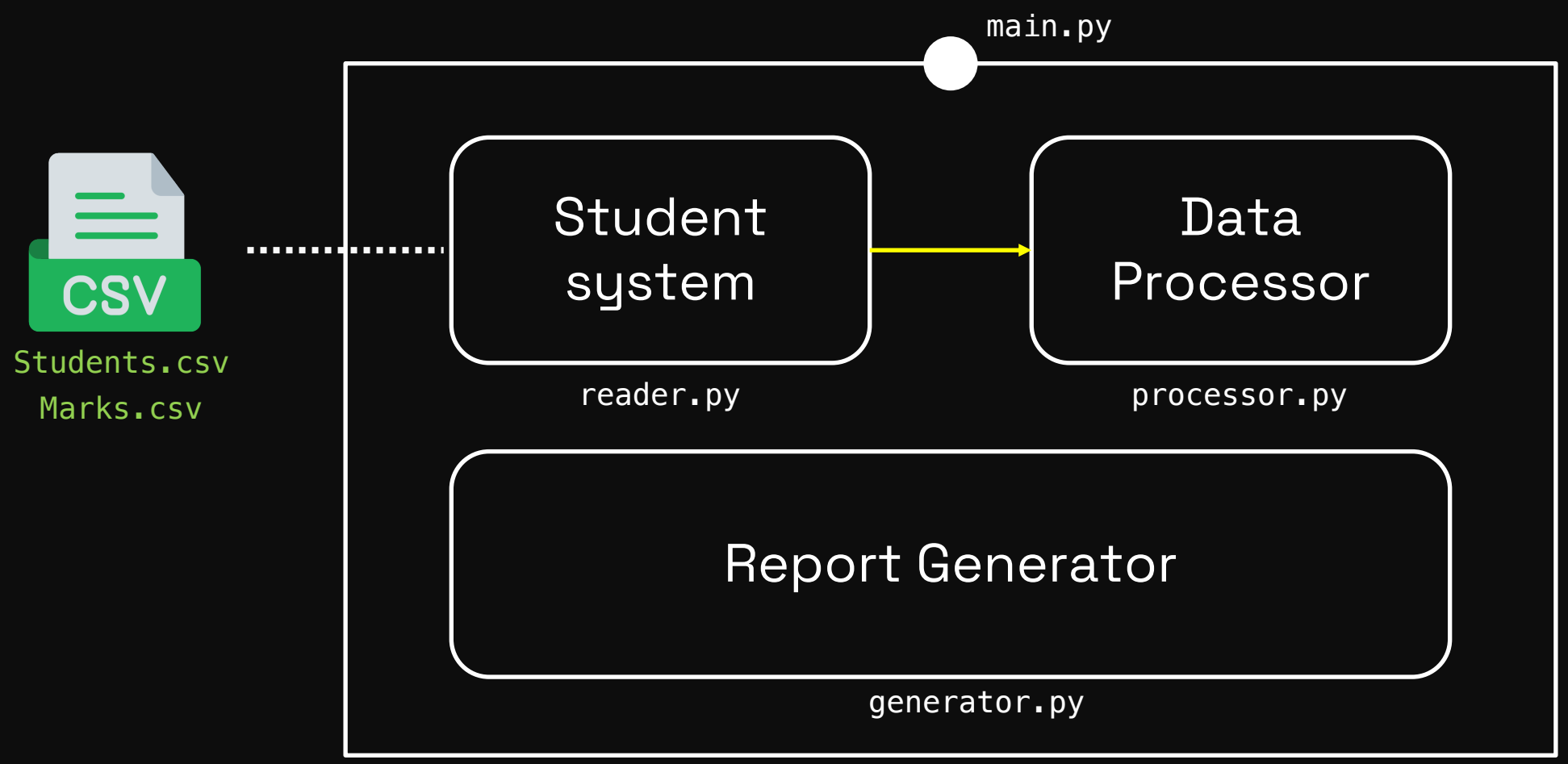

Student Report System
Student Management Service
Absence Tracking Service
Report Management Service

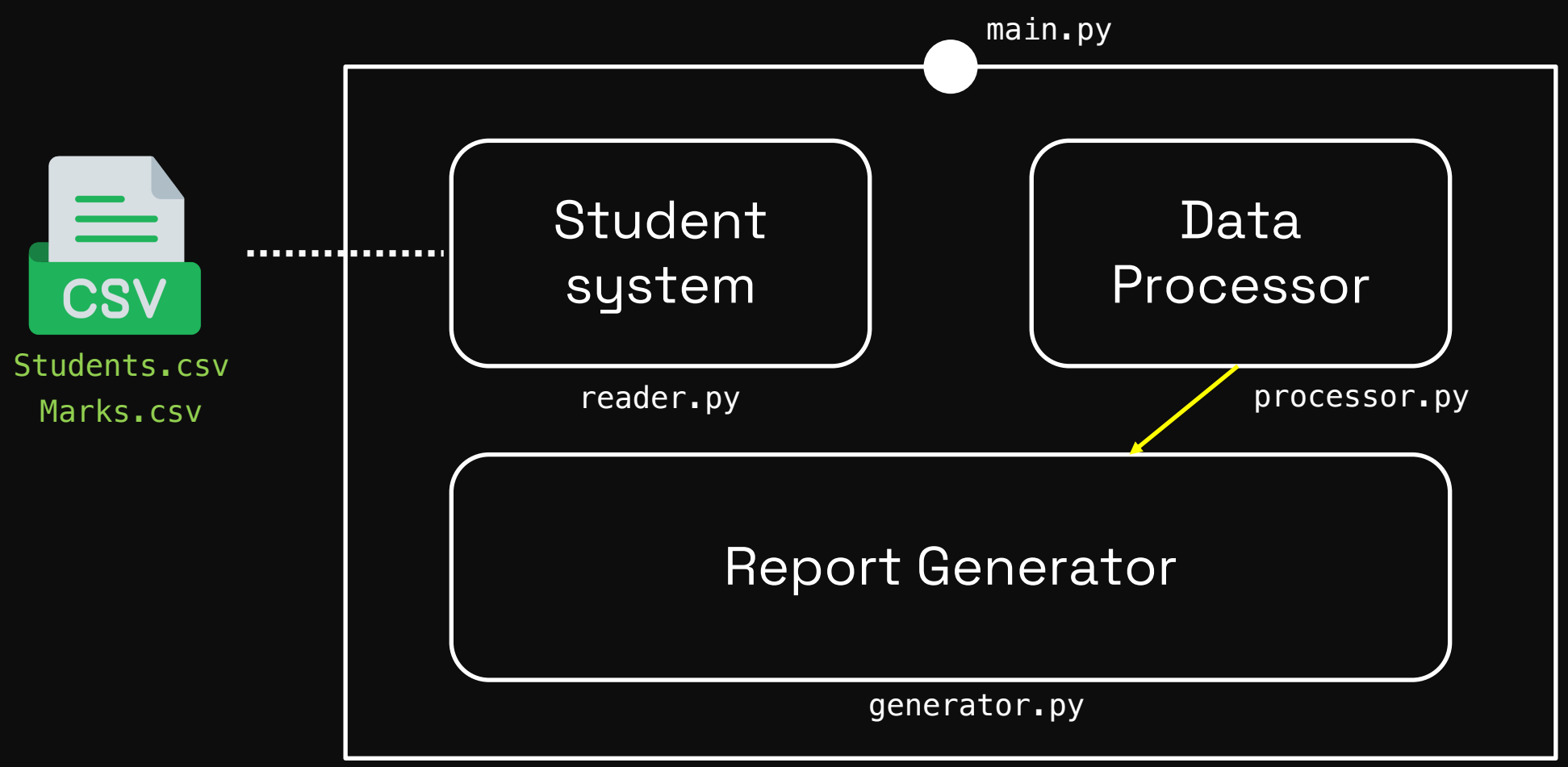
In practice: Monolith

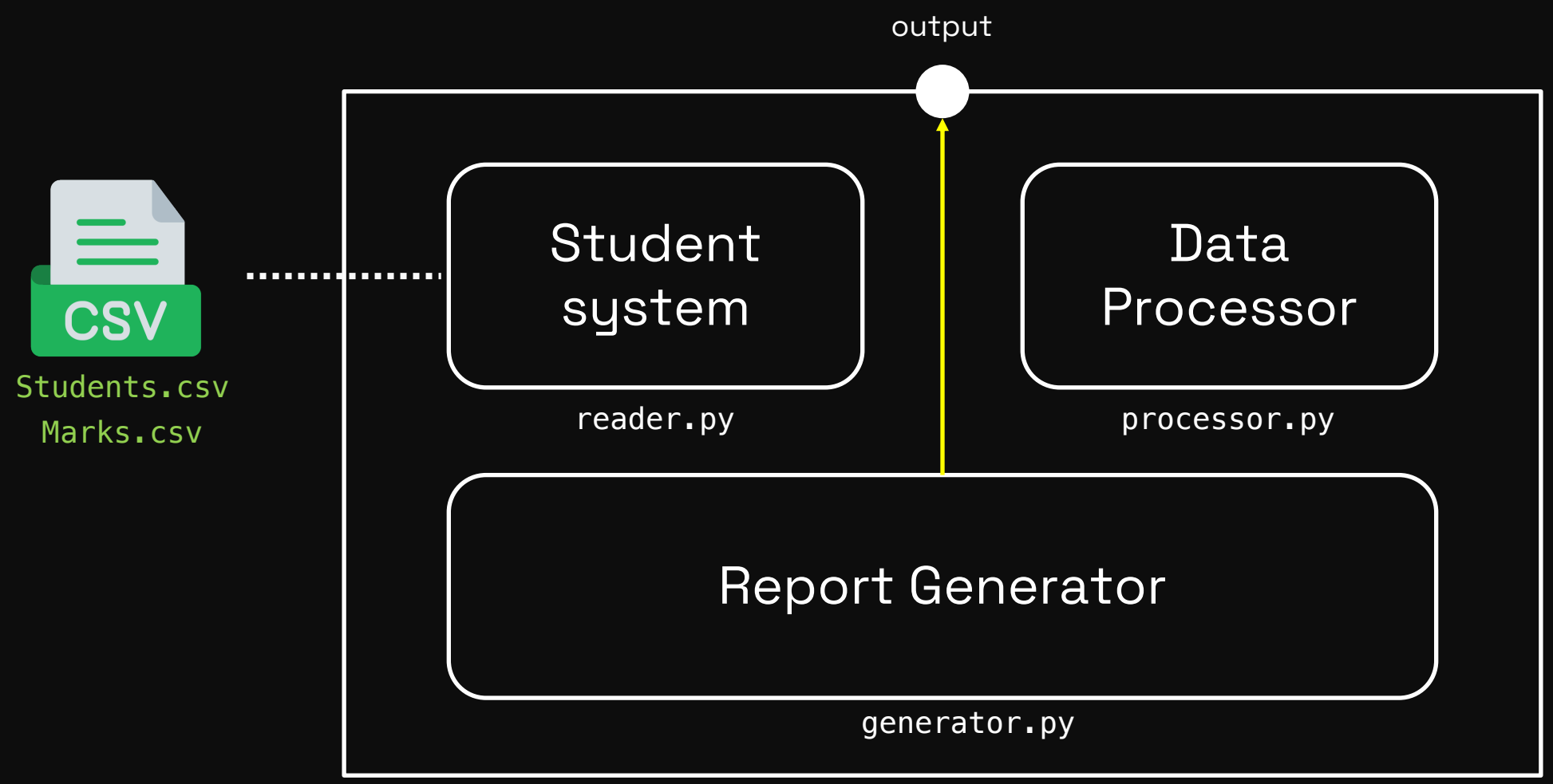



Entry point
main.py
CSV
Students.csv
Marks.csv
Student system
reader.py
Data Processor
processor.py
Report Generator
generator.py


main.py
CSV
Students.csv
Marks.csv
Student system
reader.py
Data Processor
processor.py
Report Generator
generator.py


main.py
CSV
Students.csv
Marks.csv
Student system
reader.py
Data Processor
processor.py
Report Generator
generator.py

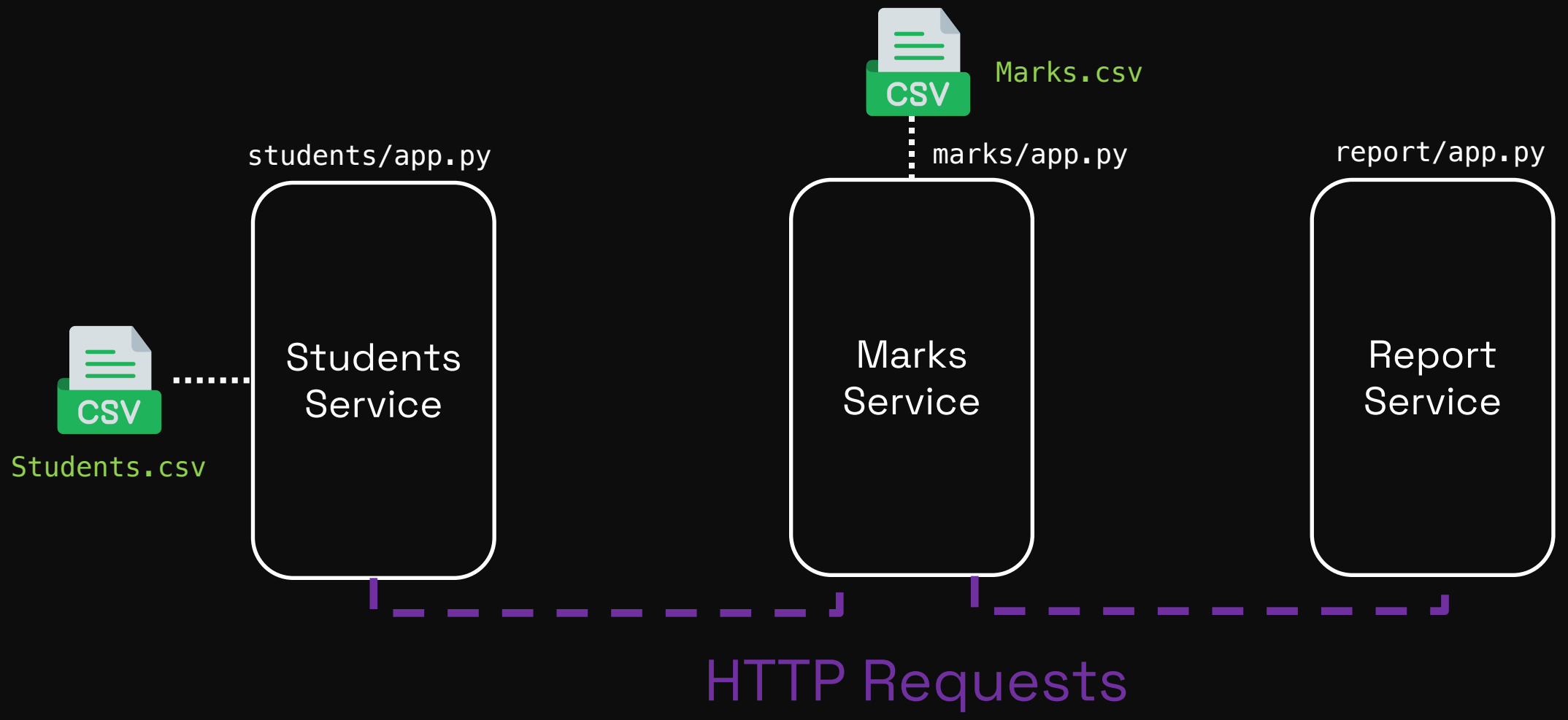

main.py
CSV
Students.csv
Marks.csv
Student system
reader.py
Data Processor
processor.py
Report Generator
generator.py

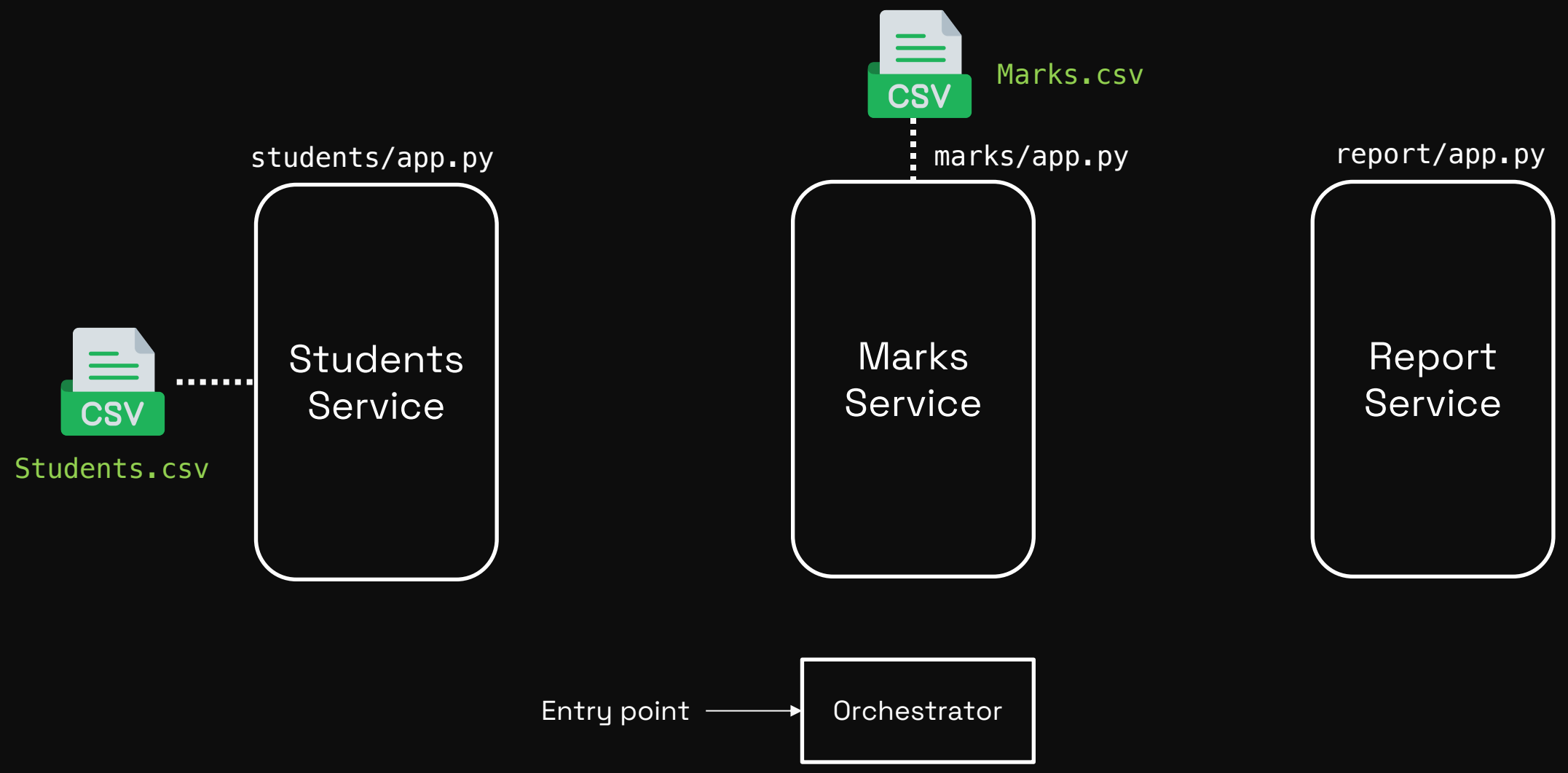

output
CSV
Students.csv
Marks.csv
Student system
reader.py
Data Processor
processor.py
Report Generator
generator.py

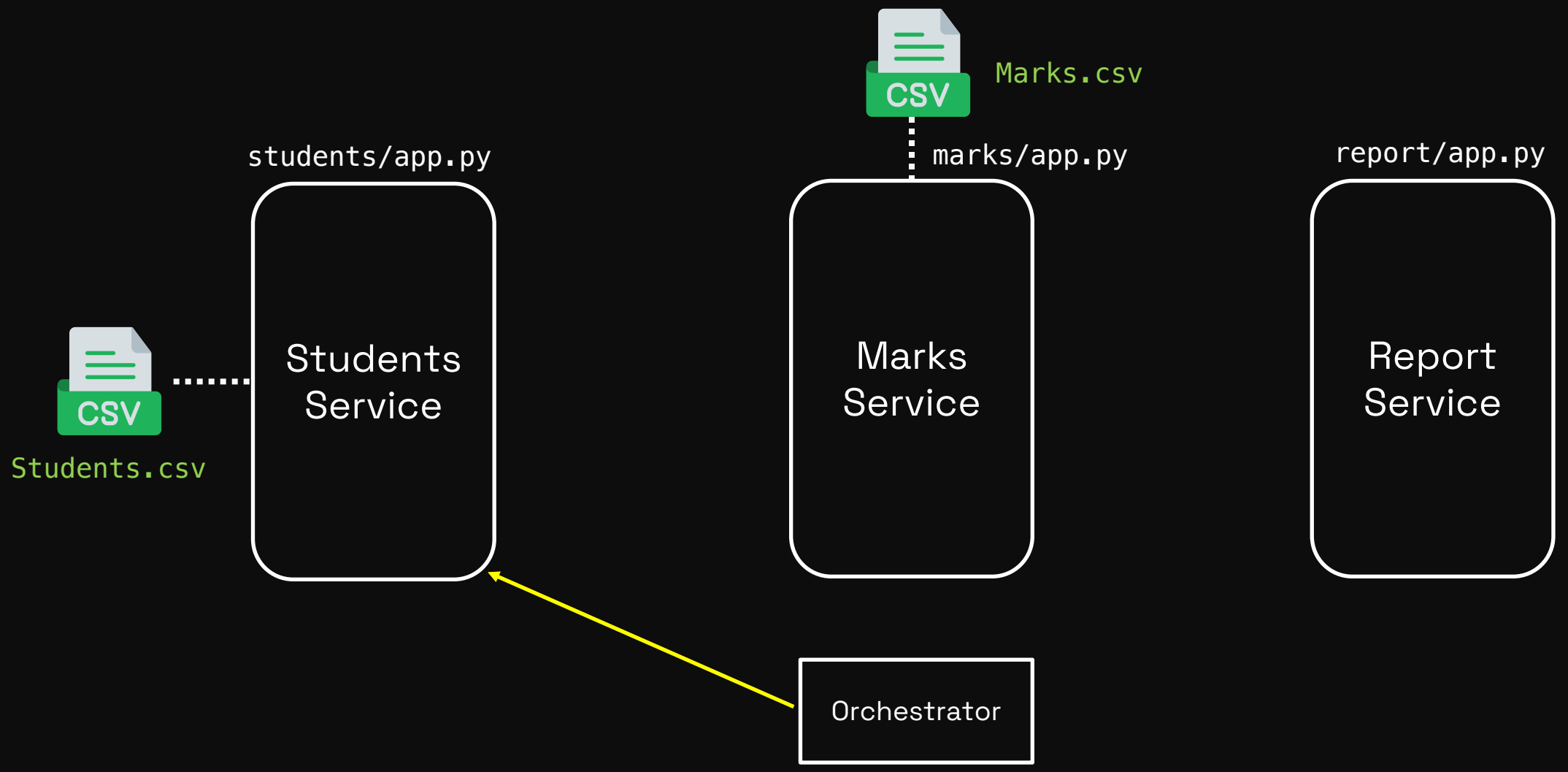
Live code

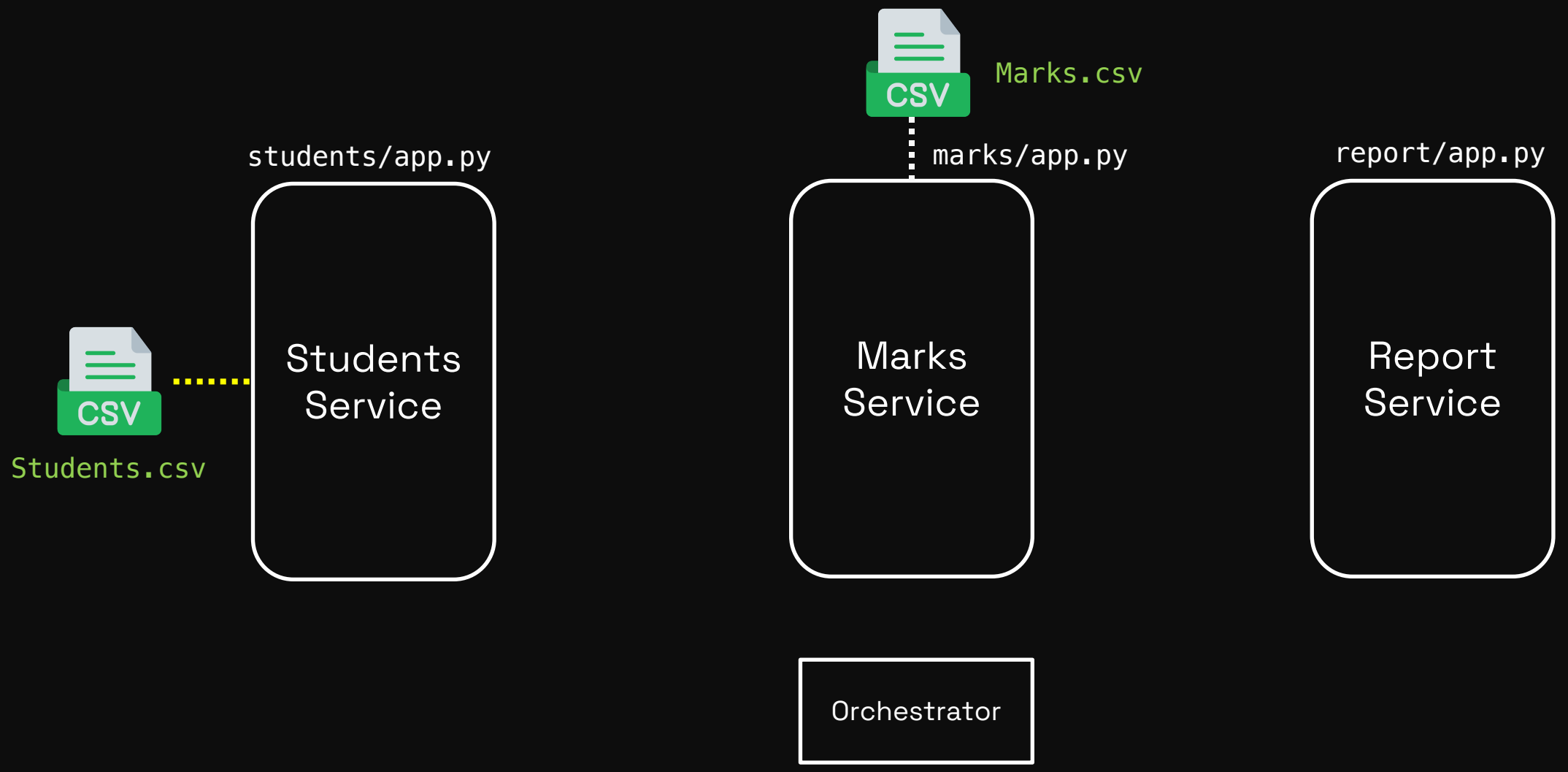
```
python main.py --student John Doe
```

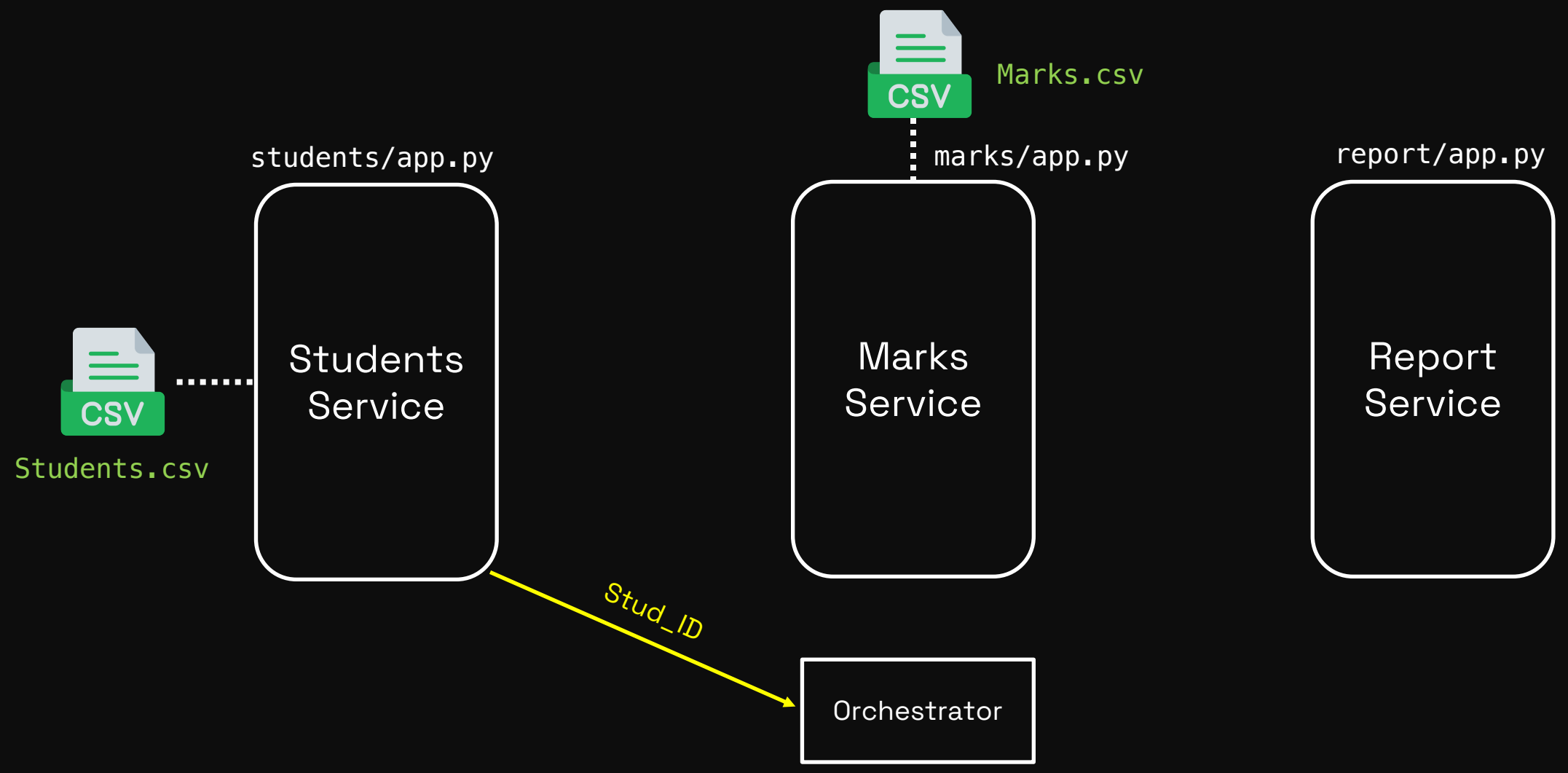
Microservices architecture

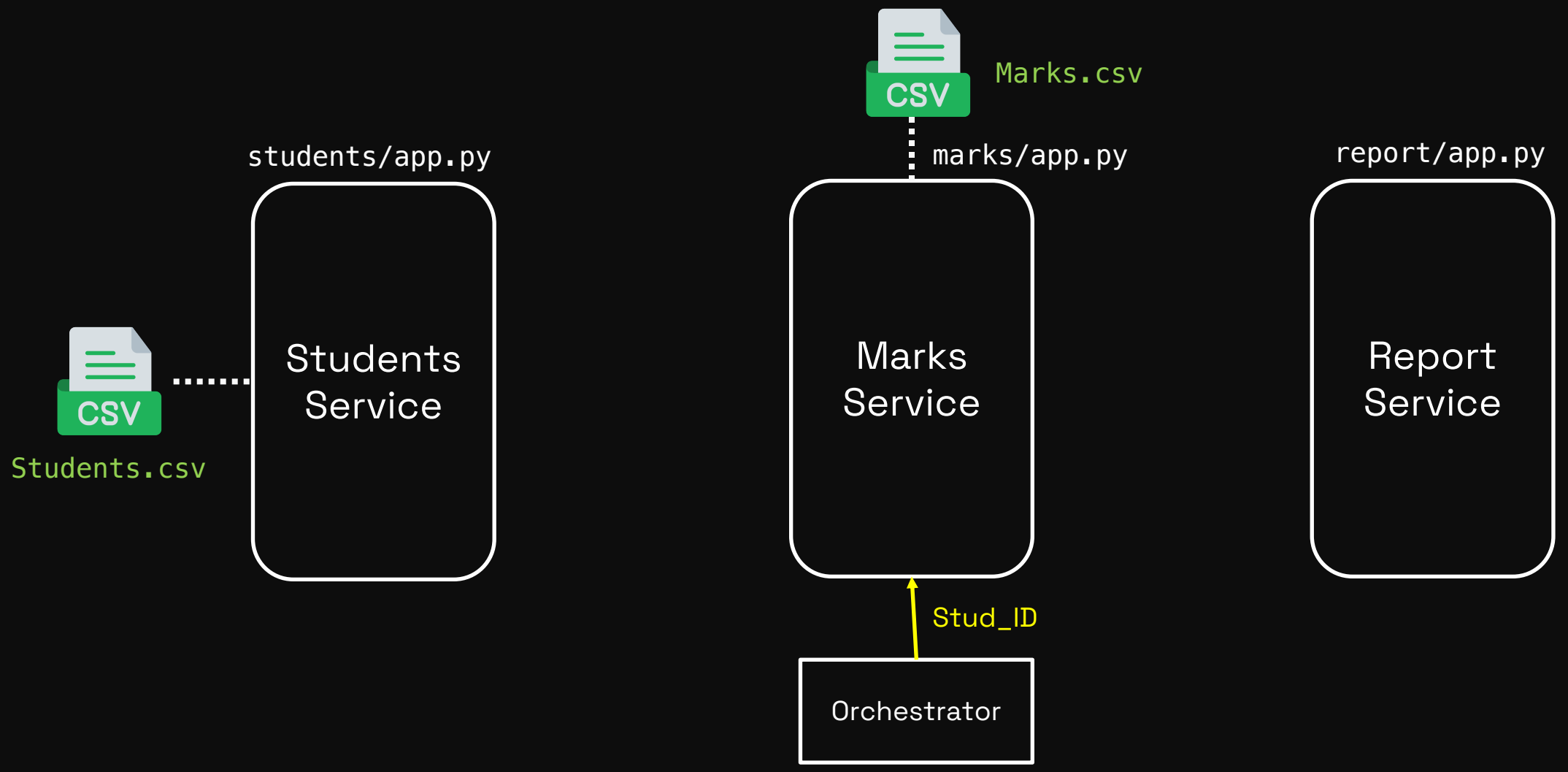


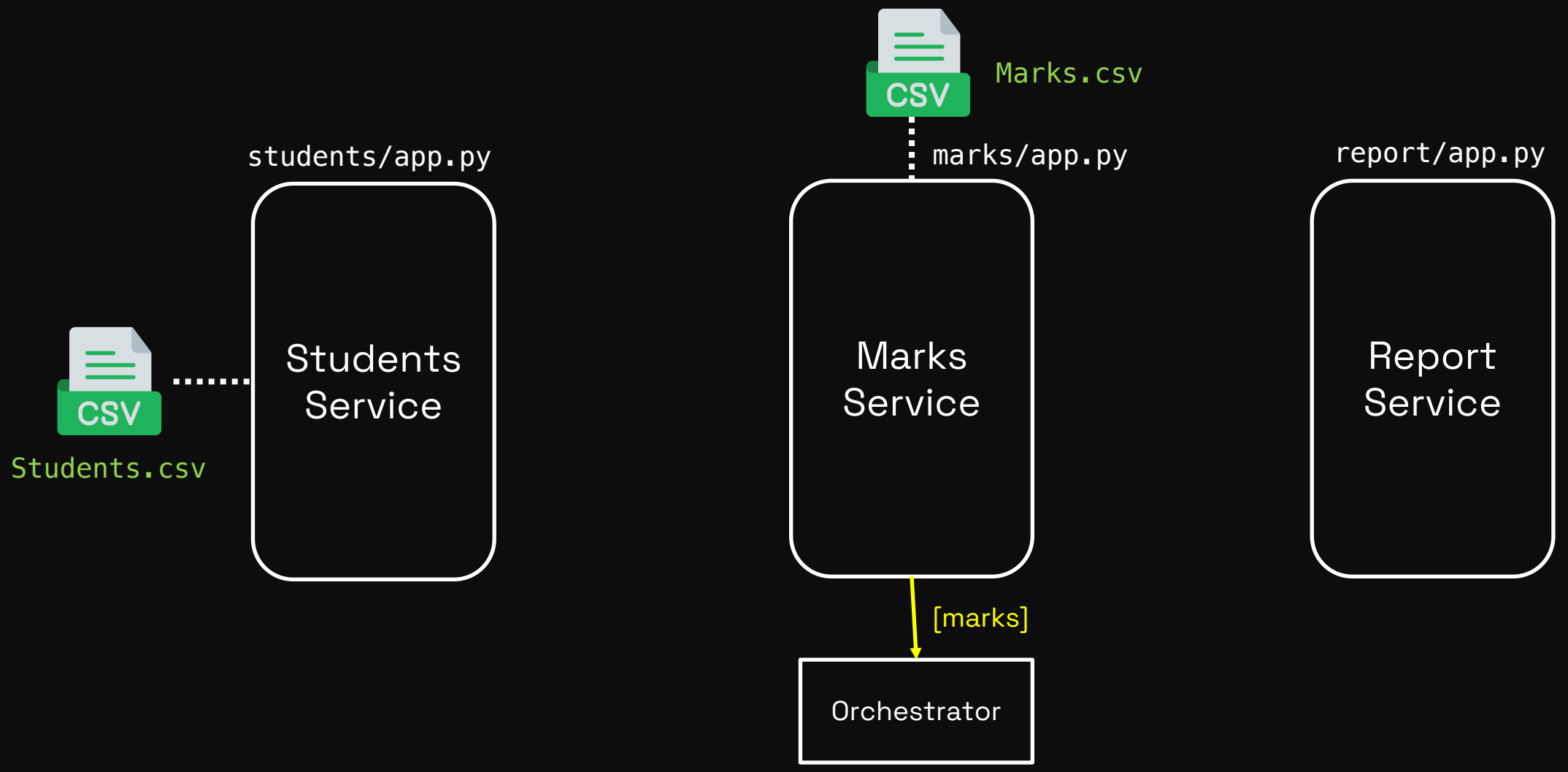

CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
Entry point
Orchestrator

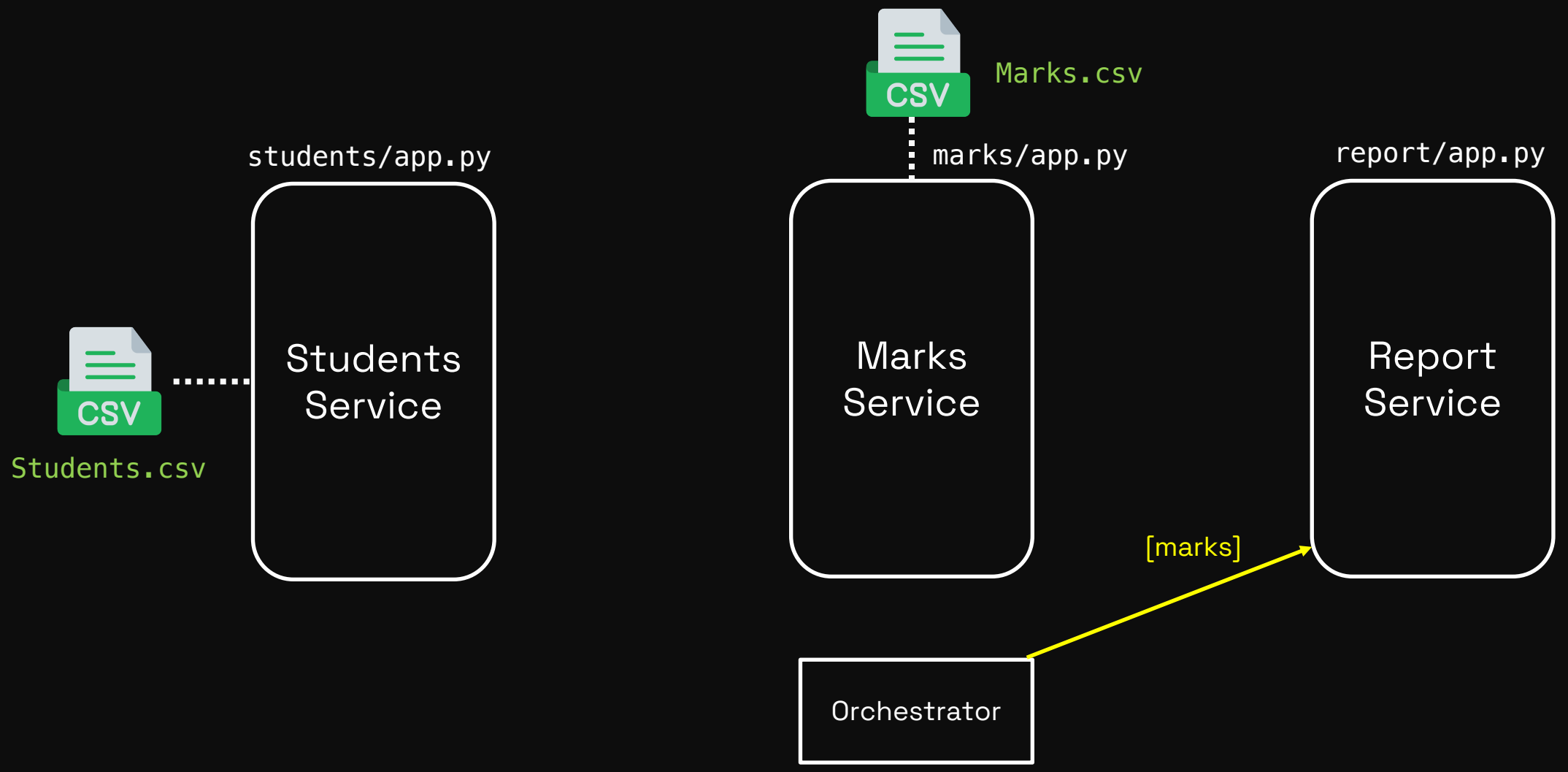

CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
Orchestrator

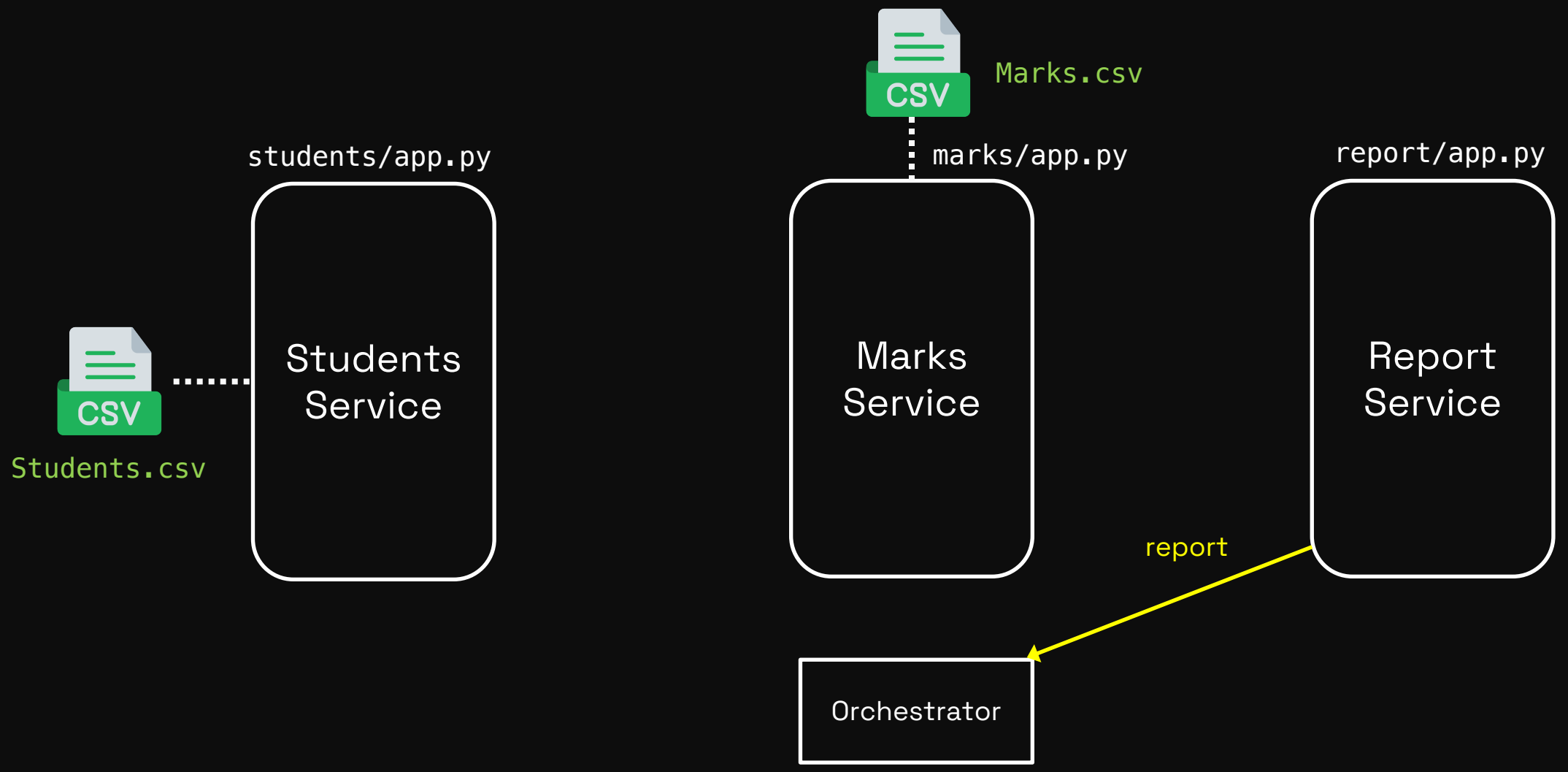

CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
Orchestrator

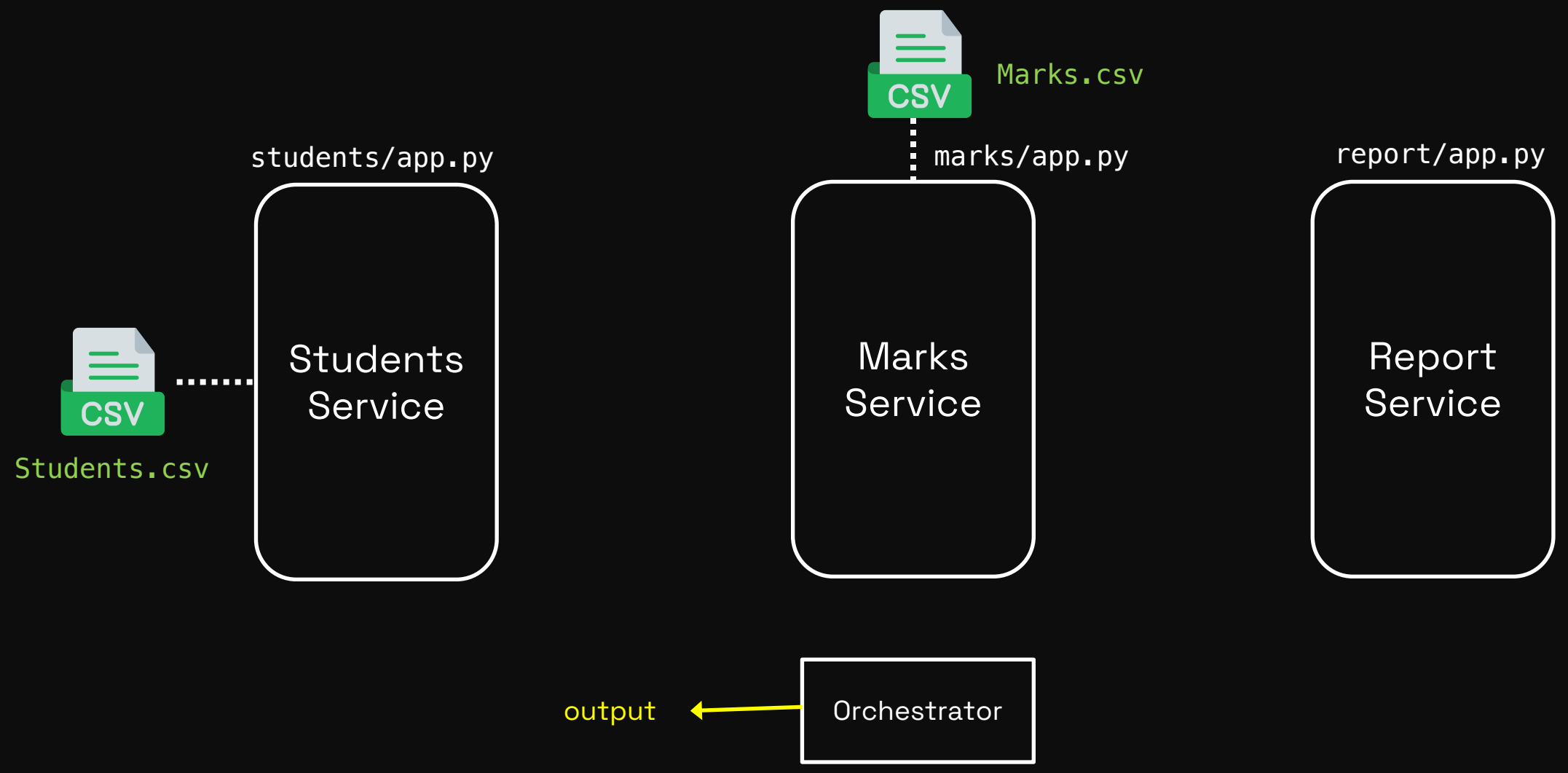

CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
Stud_ID
Orchestrator


CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
Stud_ID
Orchestrator


CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
[marks]
Orchestrator


CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
[marks]
Orchestrator


CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
report
Orchestrator

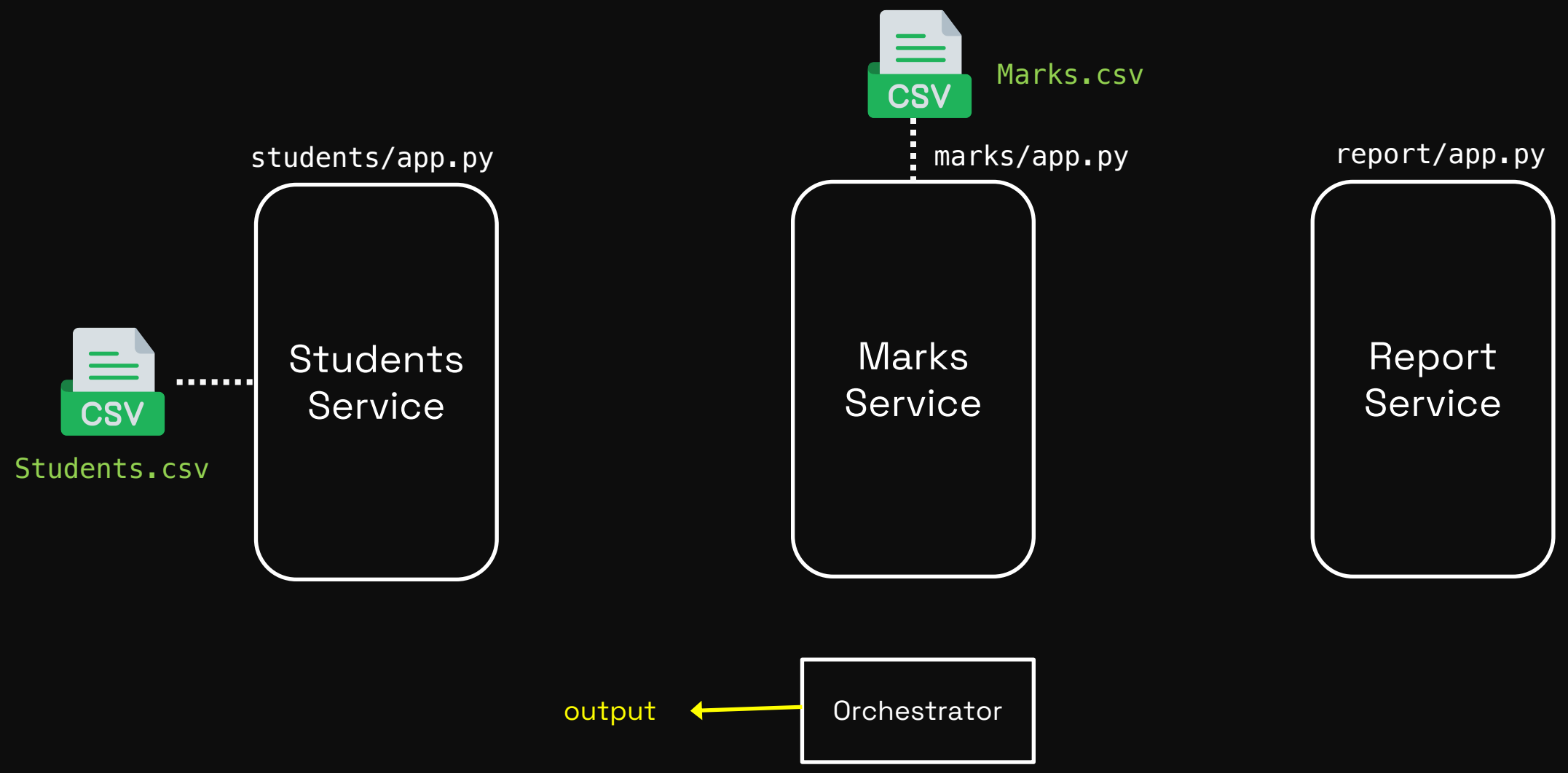

CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
output
Orchestrator

Live code

```
python main.py --student John Doe

flask run --port 5001 # start student
flask run --port 5002 # start marks
flask run --port 5003 # start report

python demo_client.py “John Doe” # start services
```


CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
output
Orchestrator

Pros and cons

- Simple
- Easy to debug/test
- No latency

- Hard to scale selectively
- Slower development over time
- Single point of failure

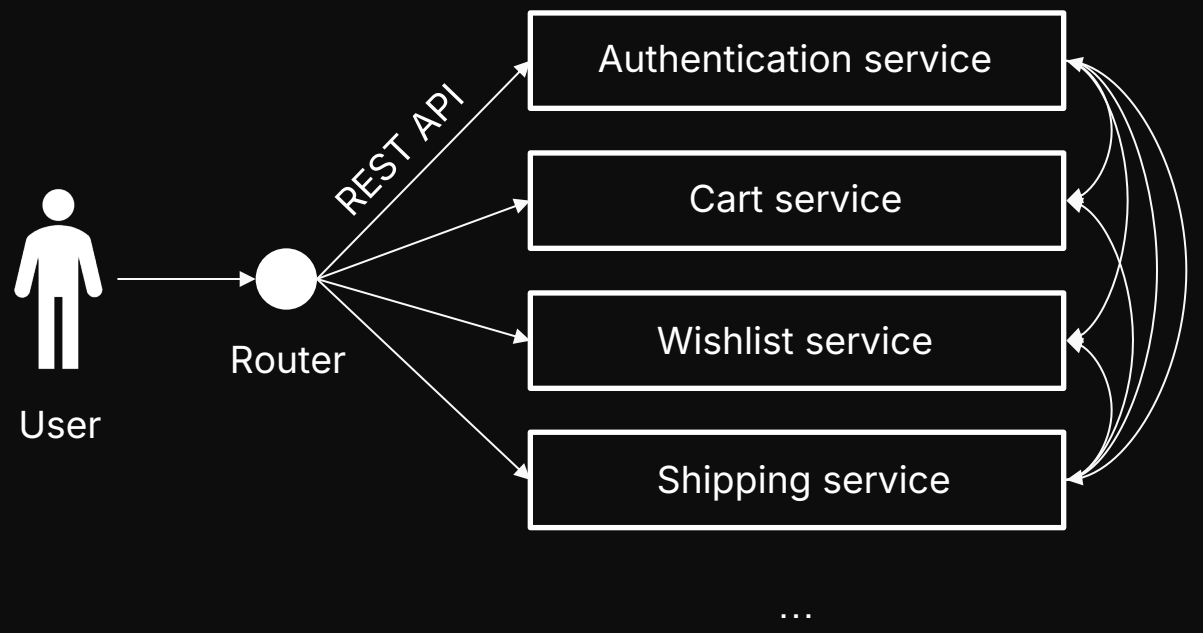
Monolith

- Independent scalability
- Independent development
- Fault isolation

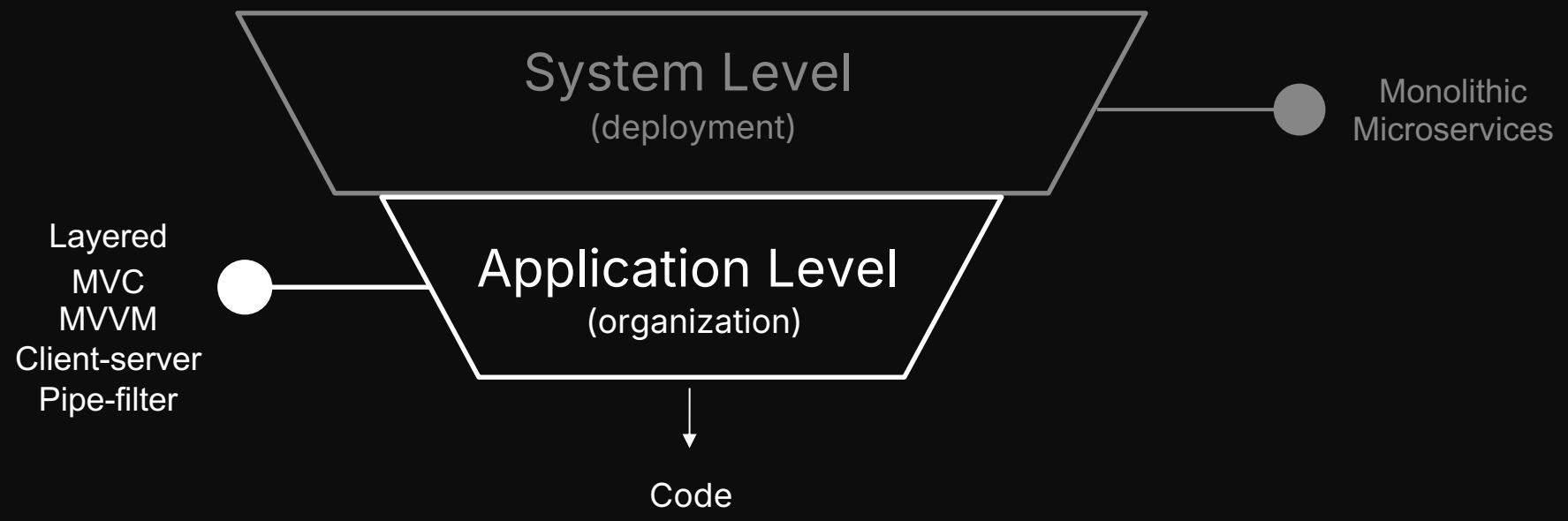
- Higher complexity
- Latency
- Requires DevOps maturity

Microservices

amazon


User
Router
REST API
Authentication service
Cart service
Wishlist service
Shipping service
...

Design patterns

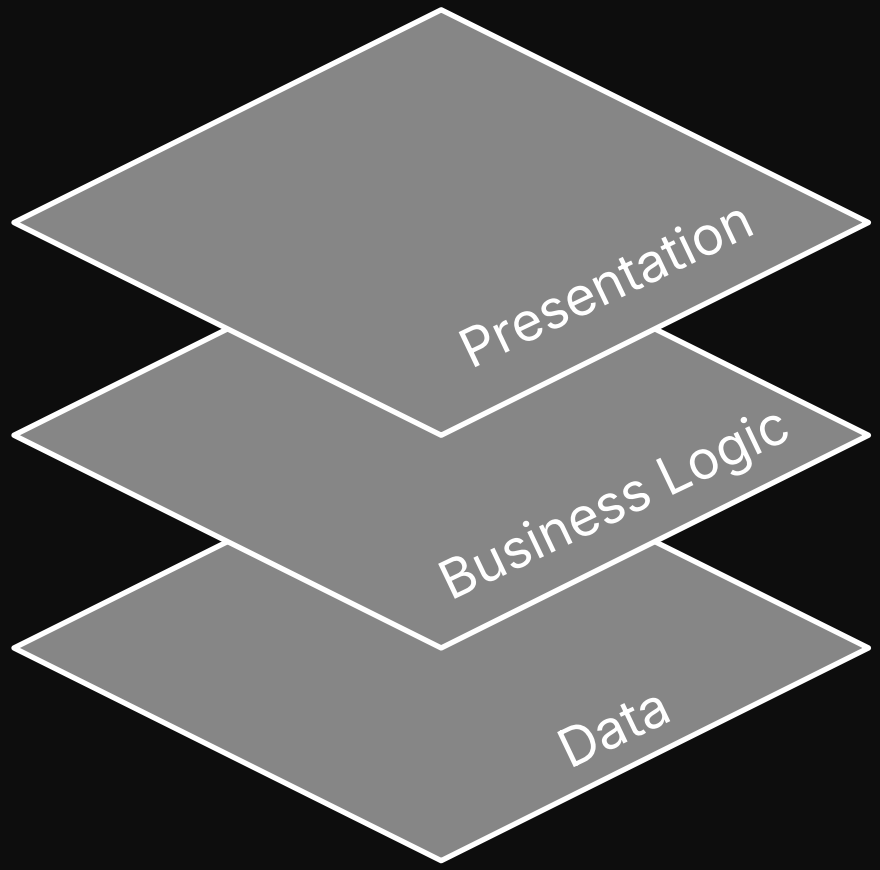


Spaghetti

“Anolini”

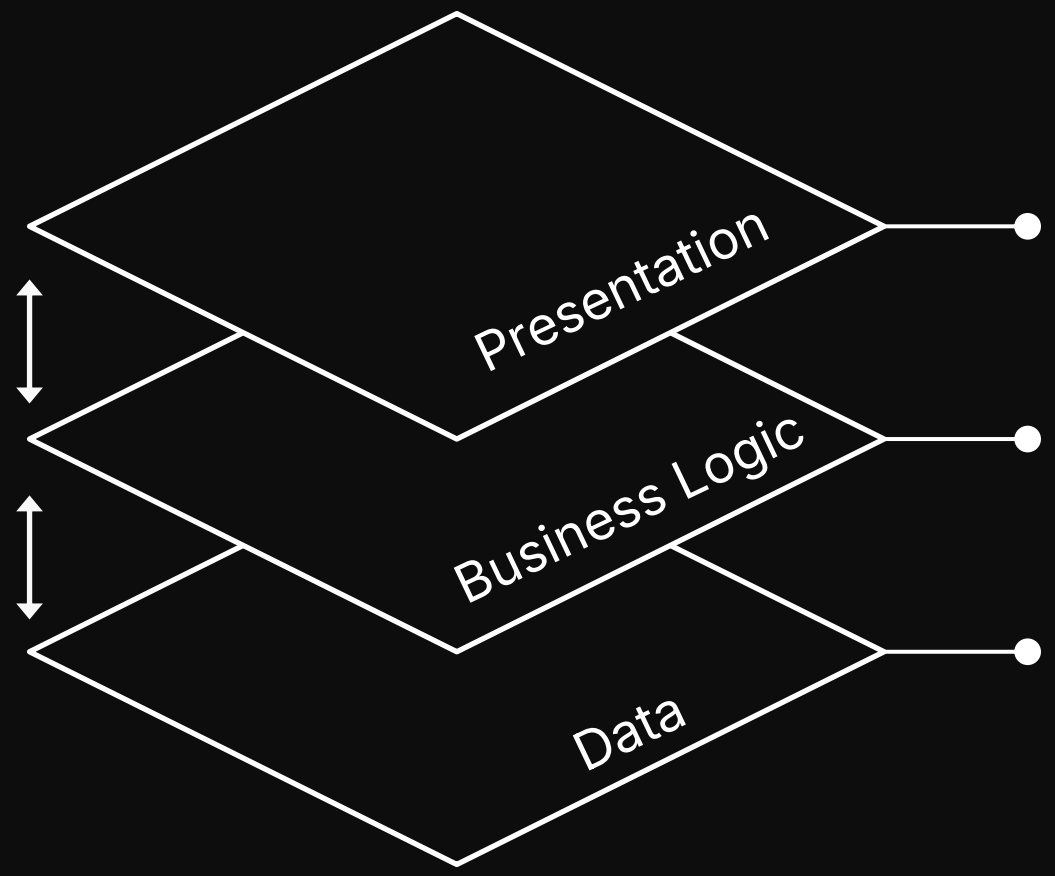
Layered architecture

- Objectives:
  - Separation
  - Independence
- Reason
  - Robustness to changes



Each layer depends on the one below and provides service to the one above

3-layered architecture

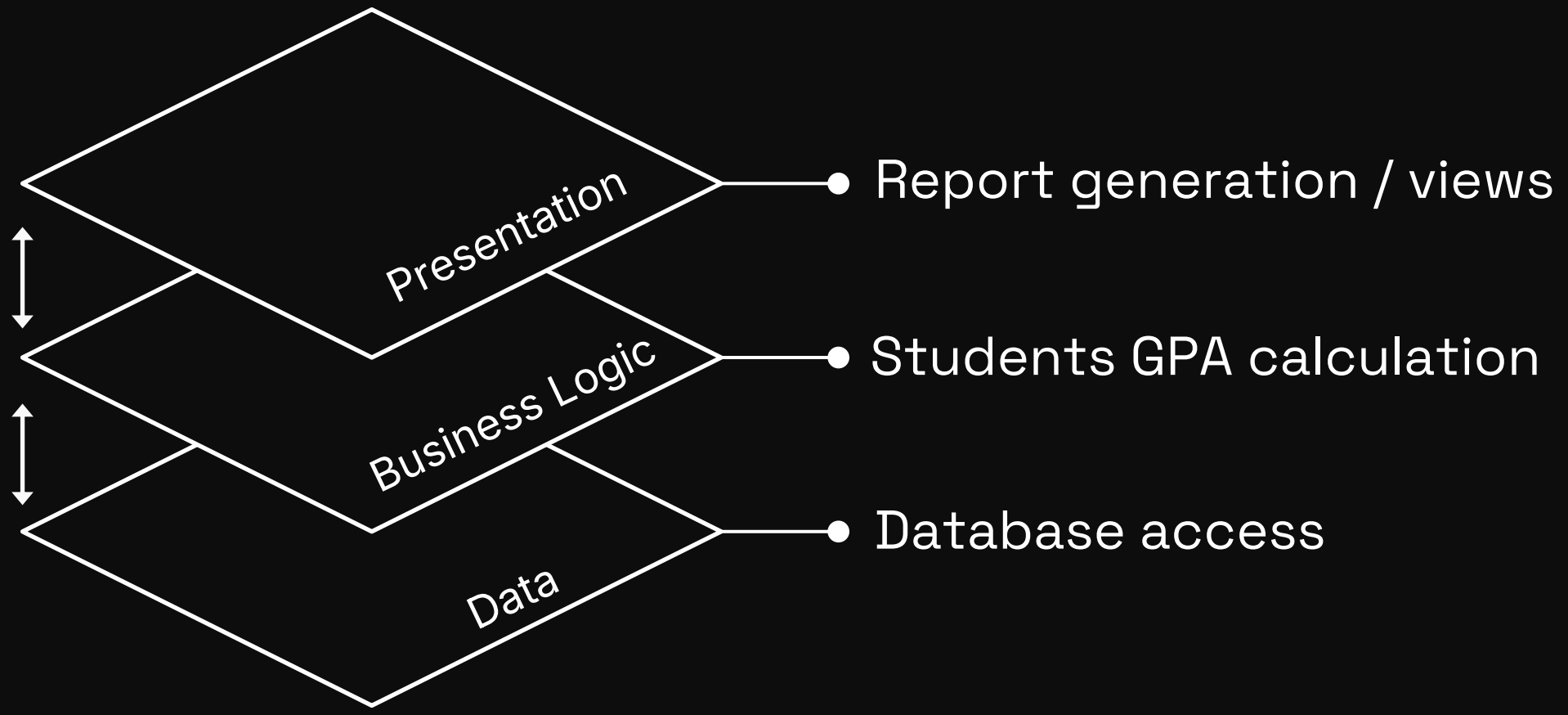


How the data is presented to the user

How the data is processed

Data management and access

In practice


Presentation
Business Logic
Data
Report generation / views
Students GPA calculation
Database access

Model-view-controller

Connection layer

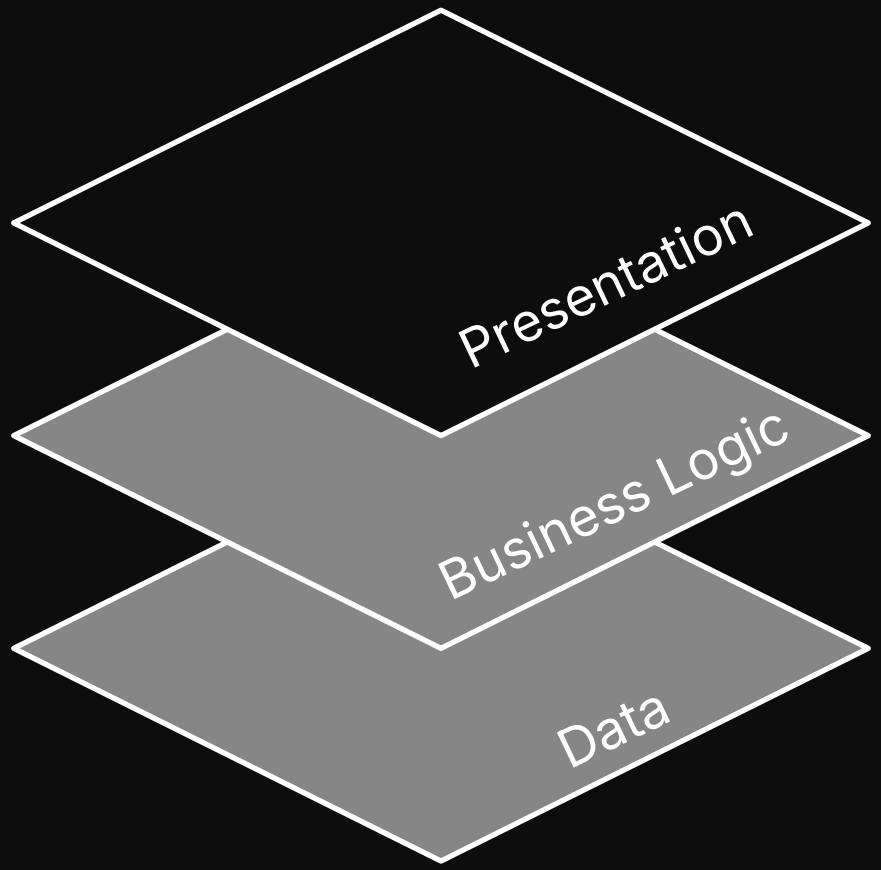
Controller

Model

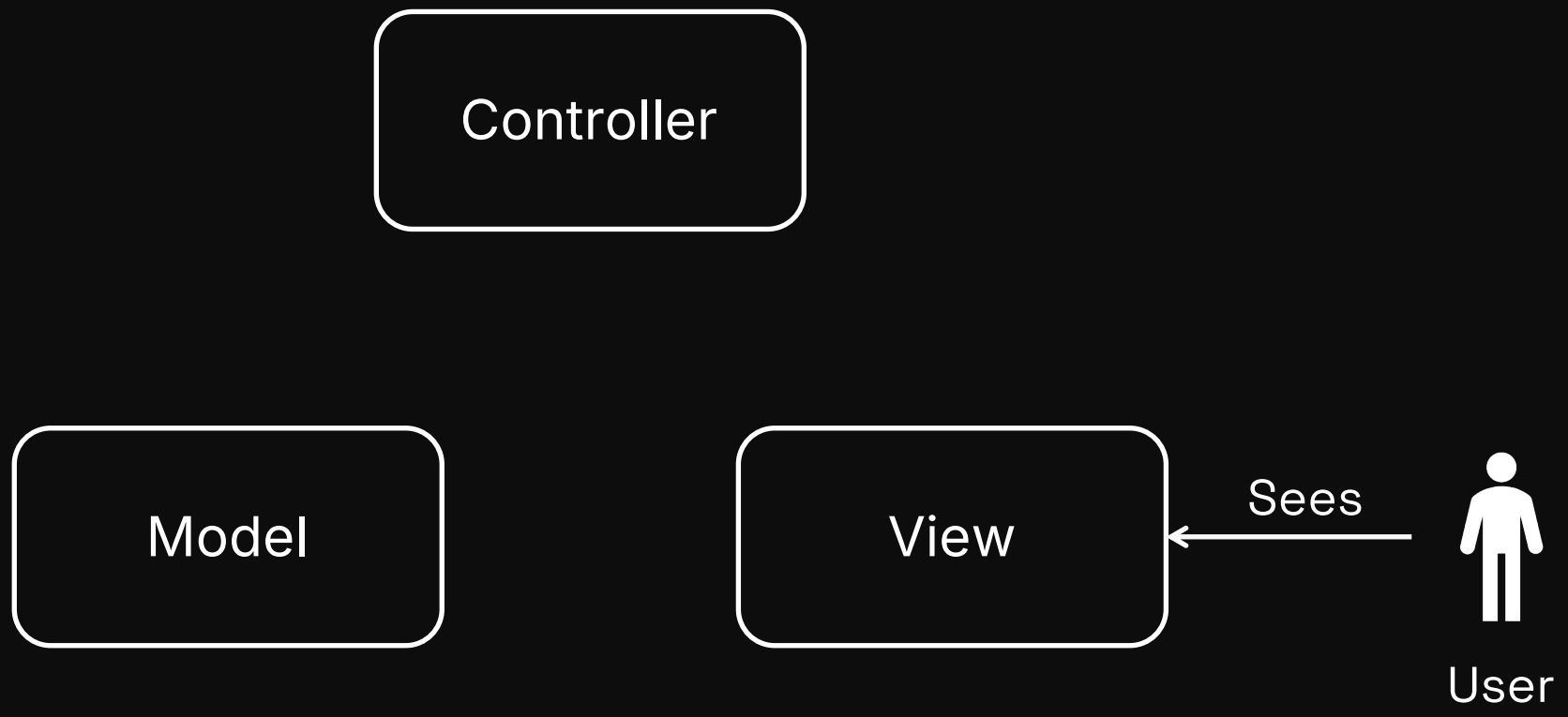
Handles the data

View

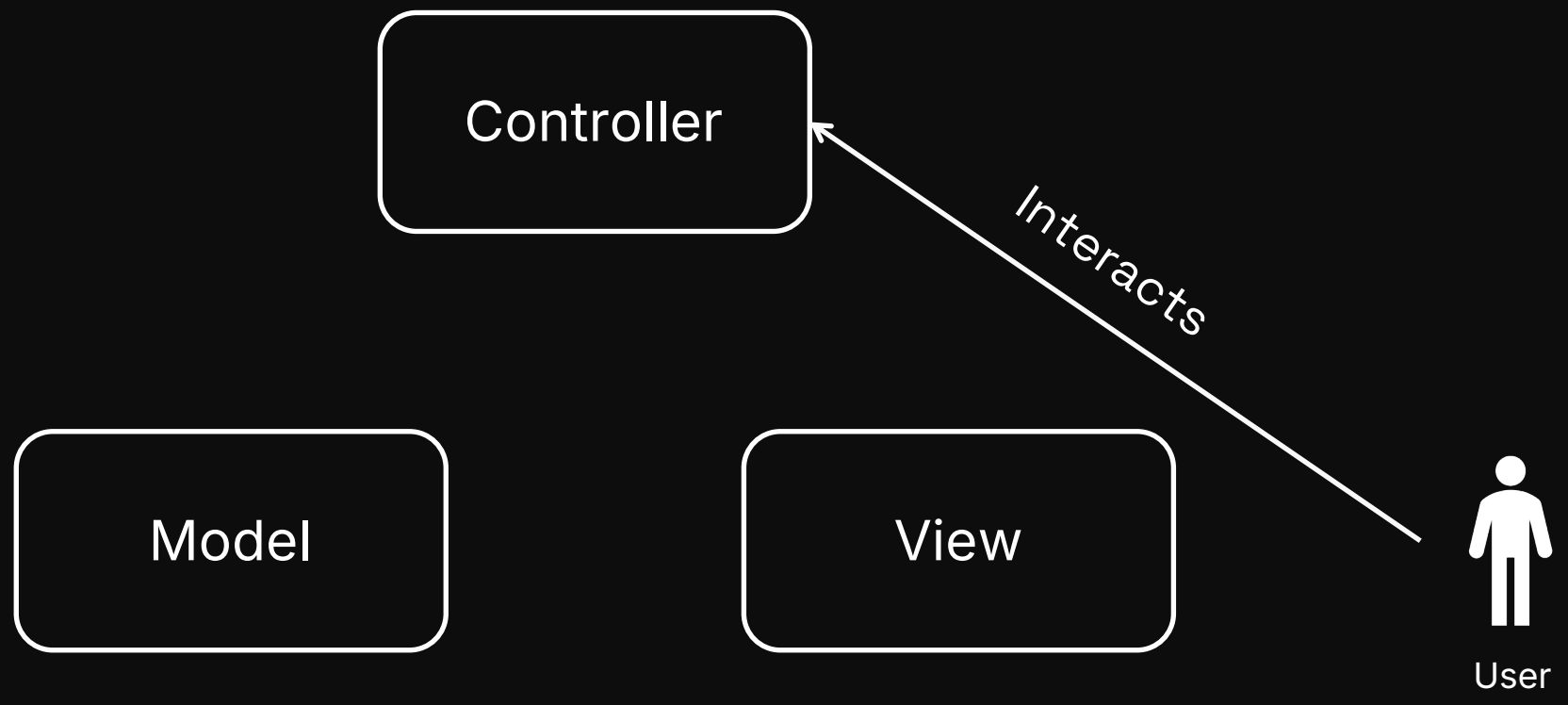
Handles the interaction with the user


Presentation
Business Logic
Data

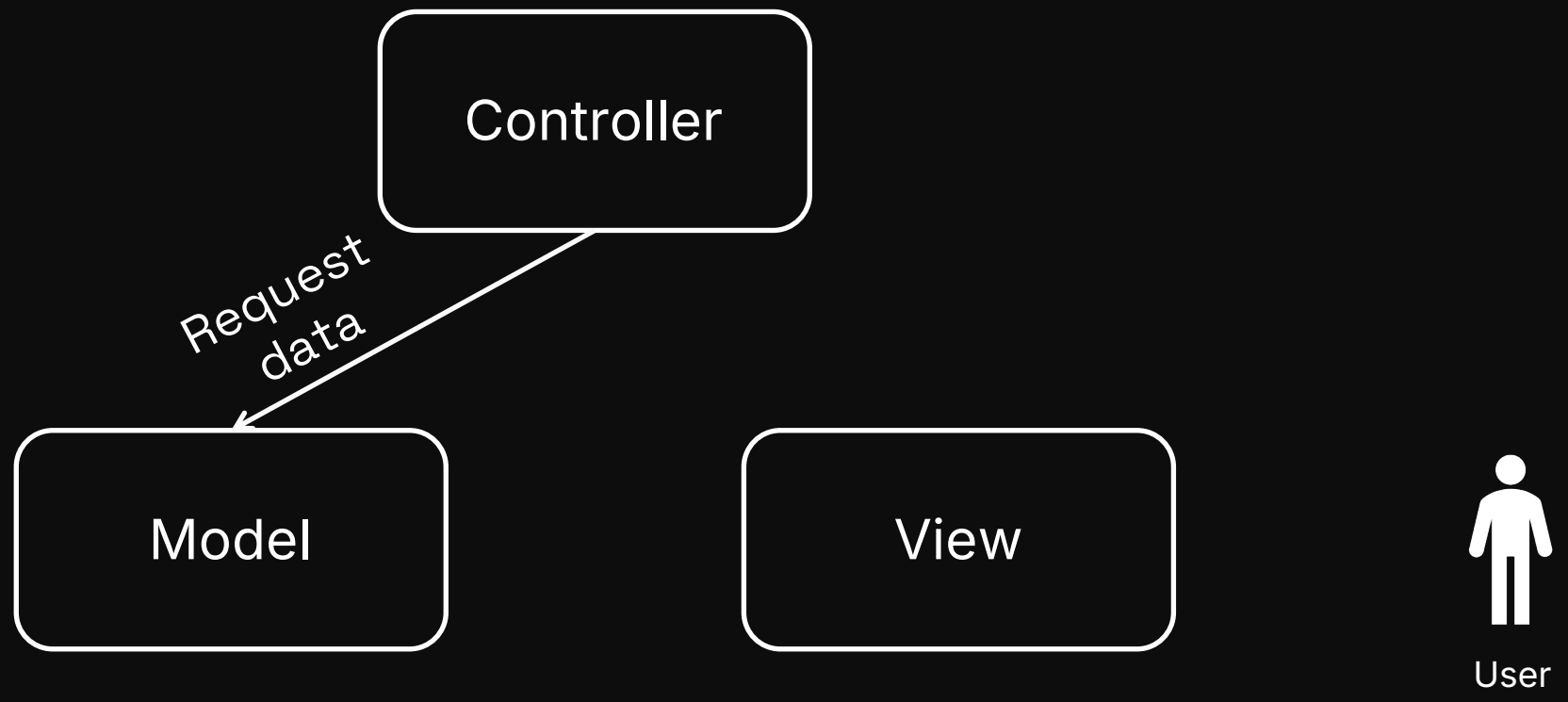
Web-based applications



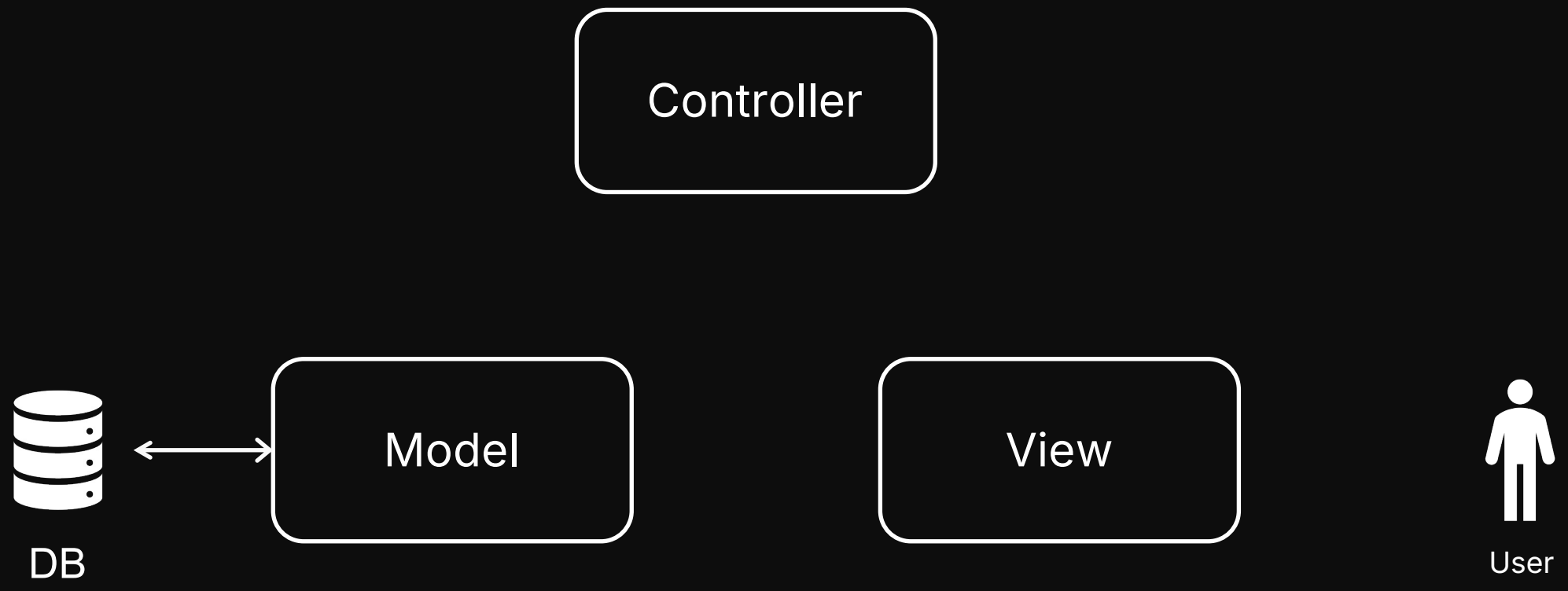
Model-view-controller



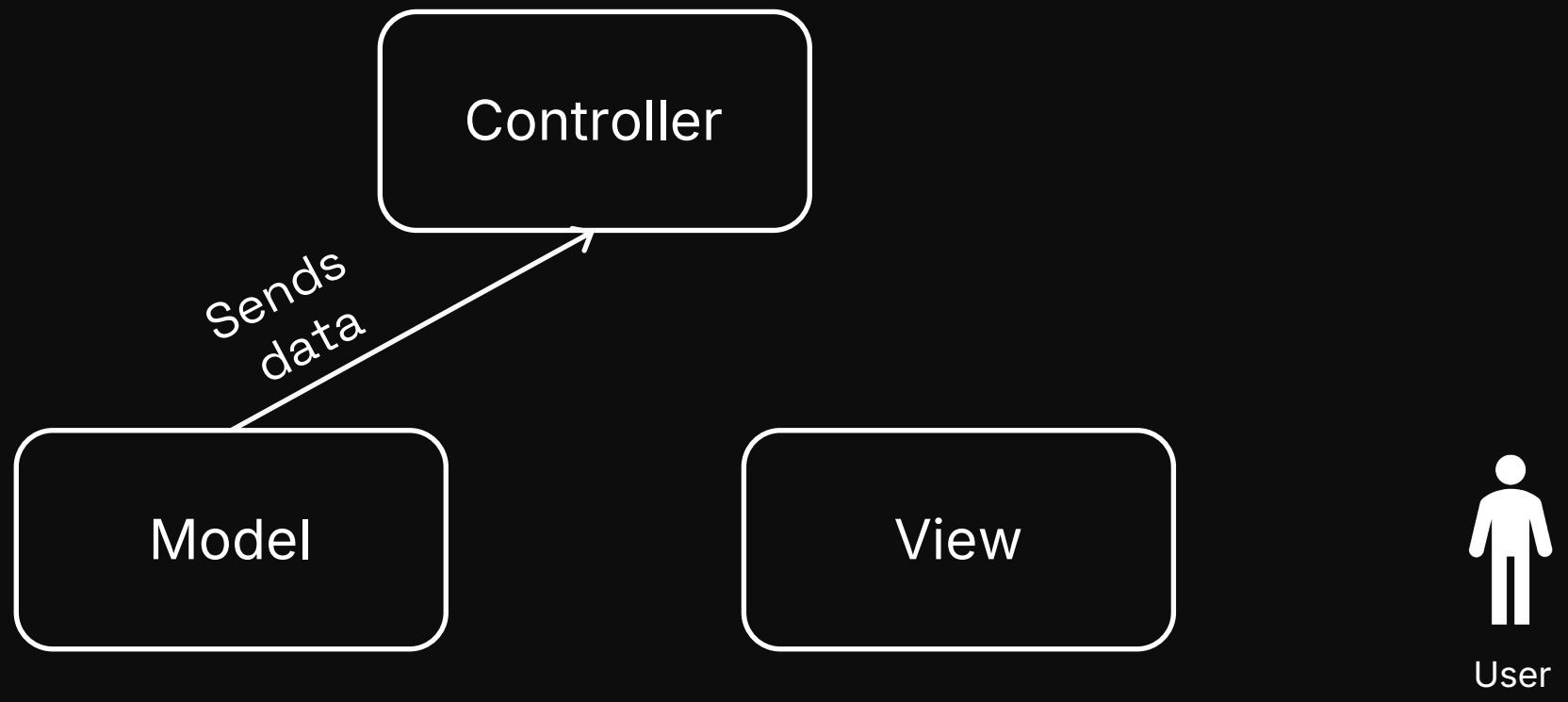
Model-view-controller



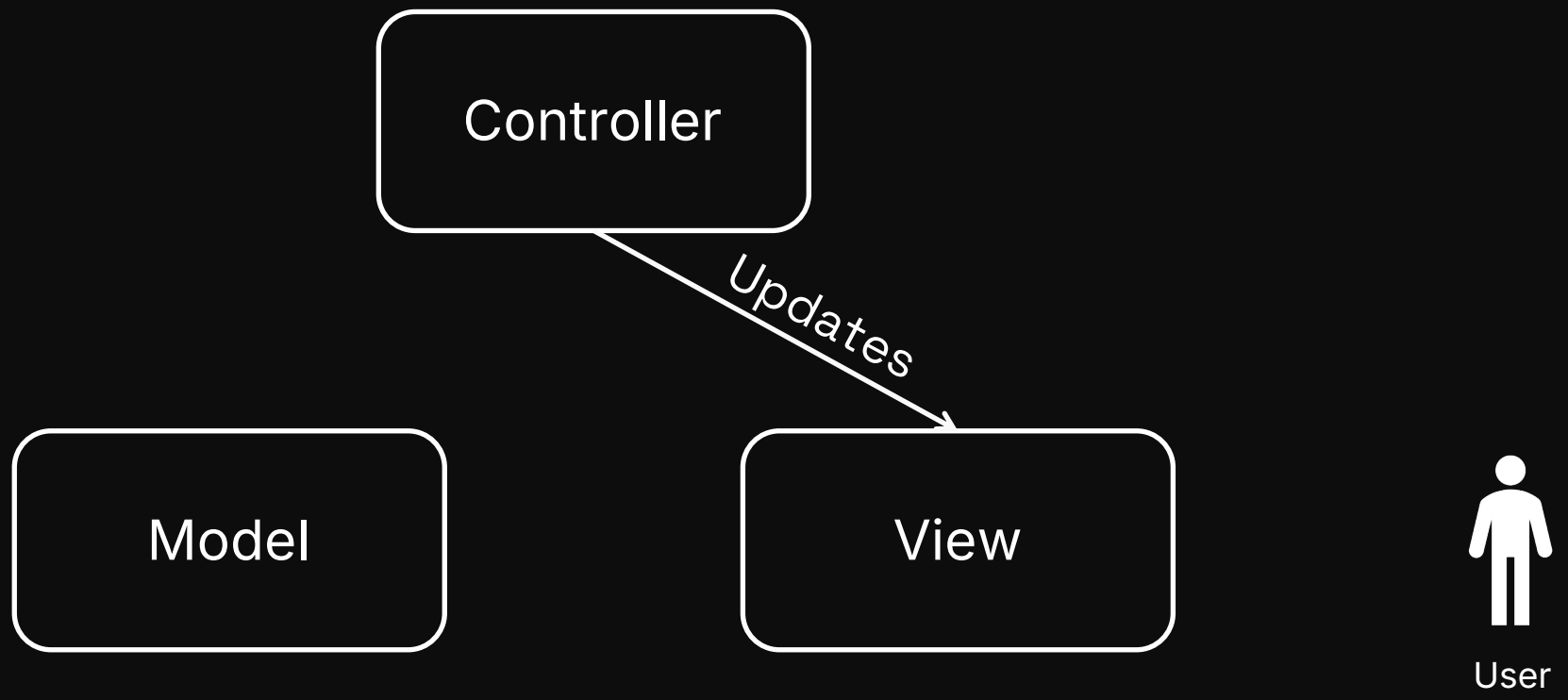
Model-view-controller



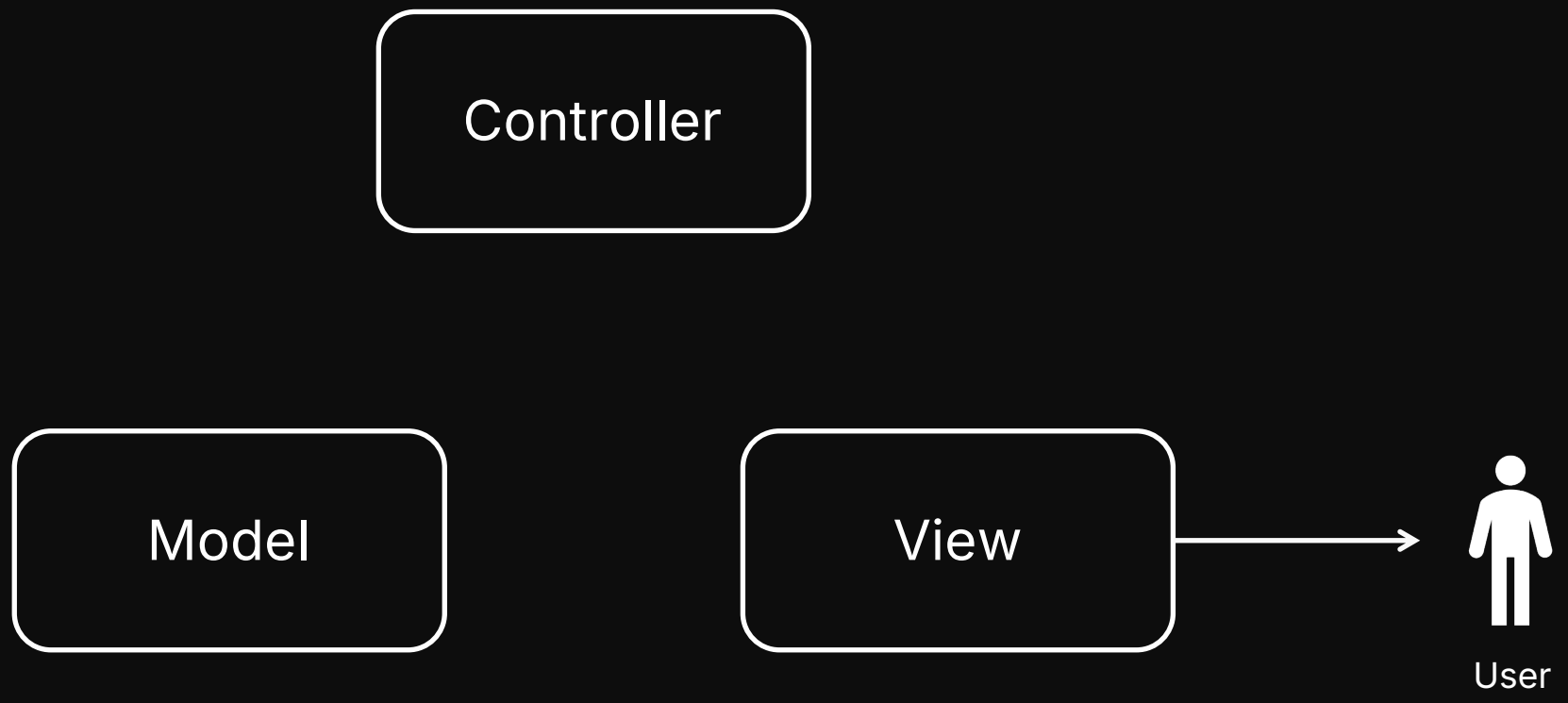
Model-view-controller



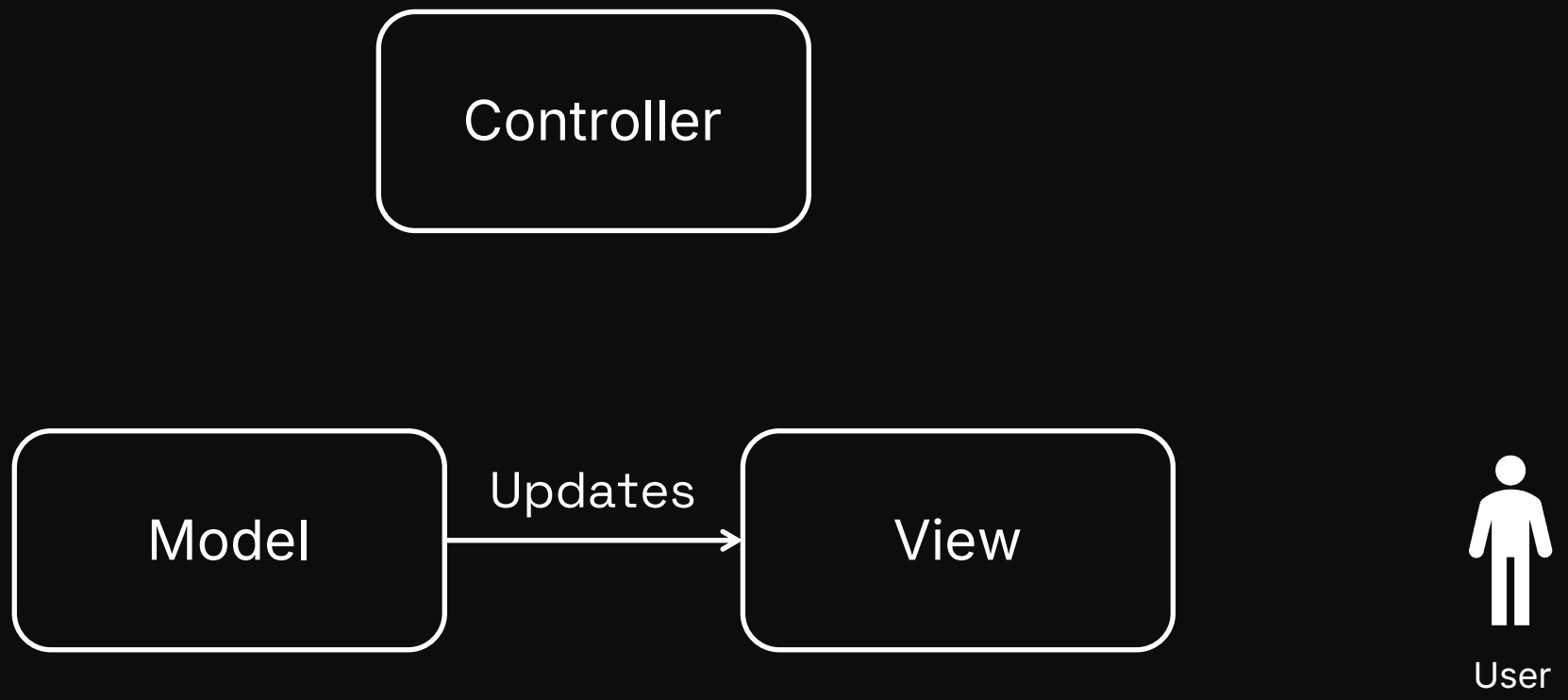
Model-view-controller



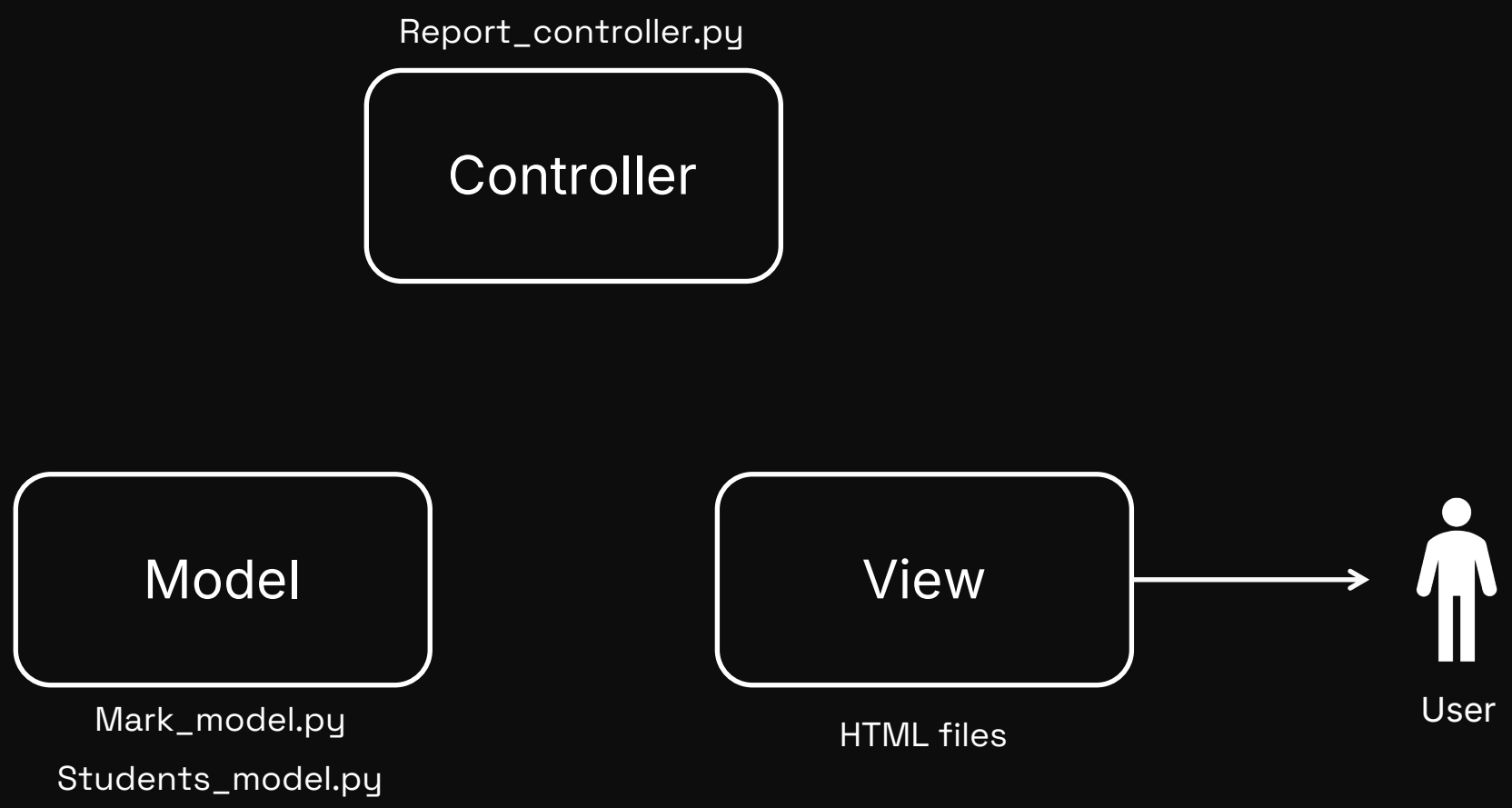
Model-view-controller



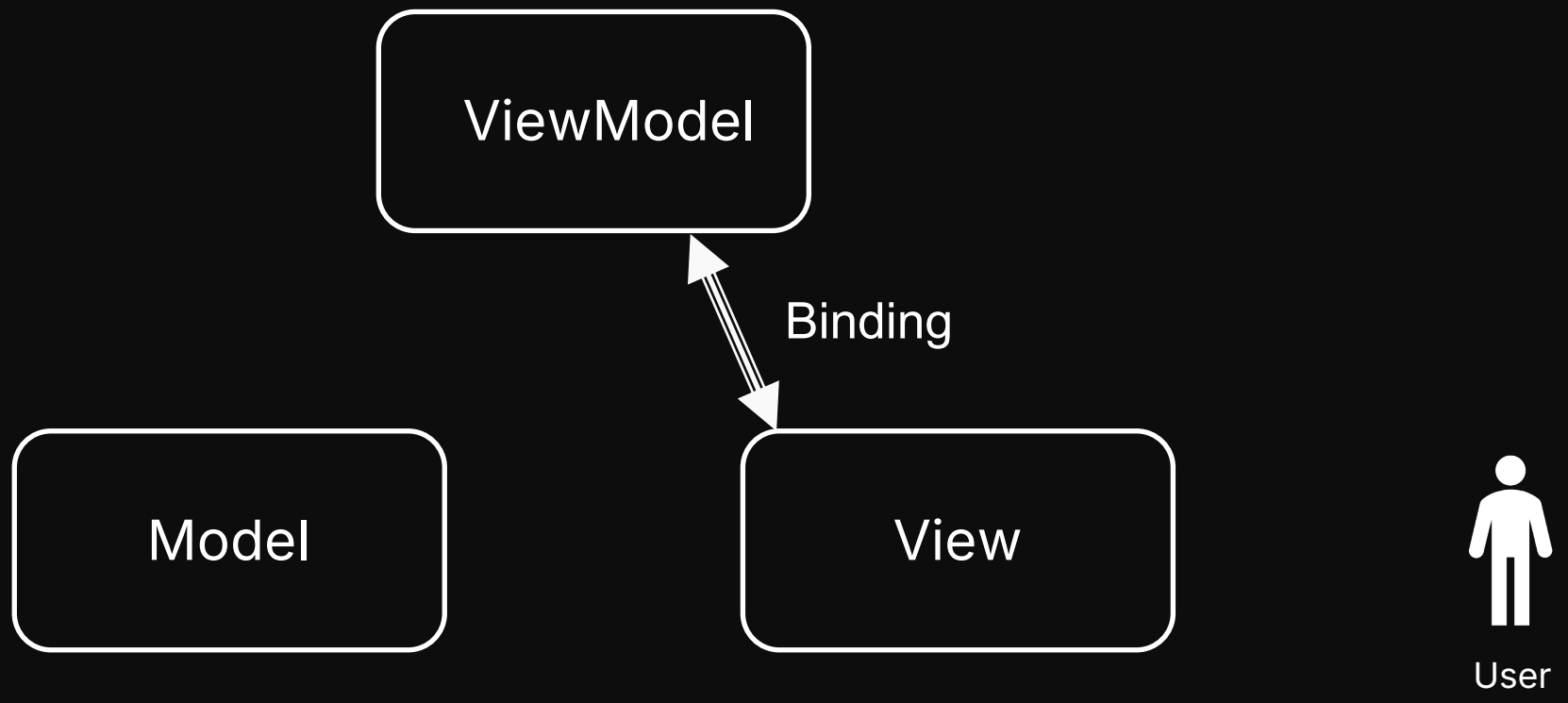
Original version

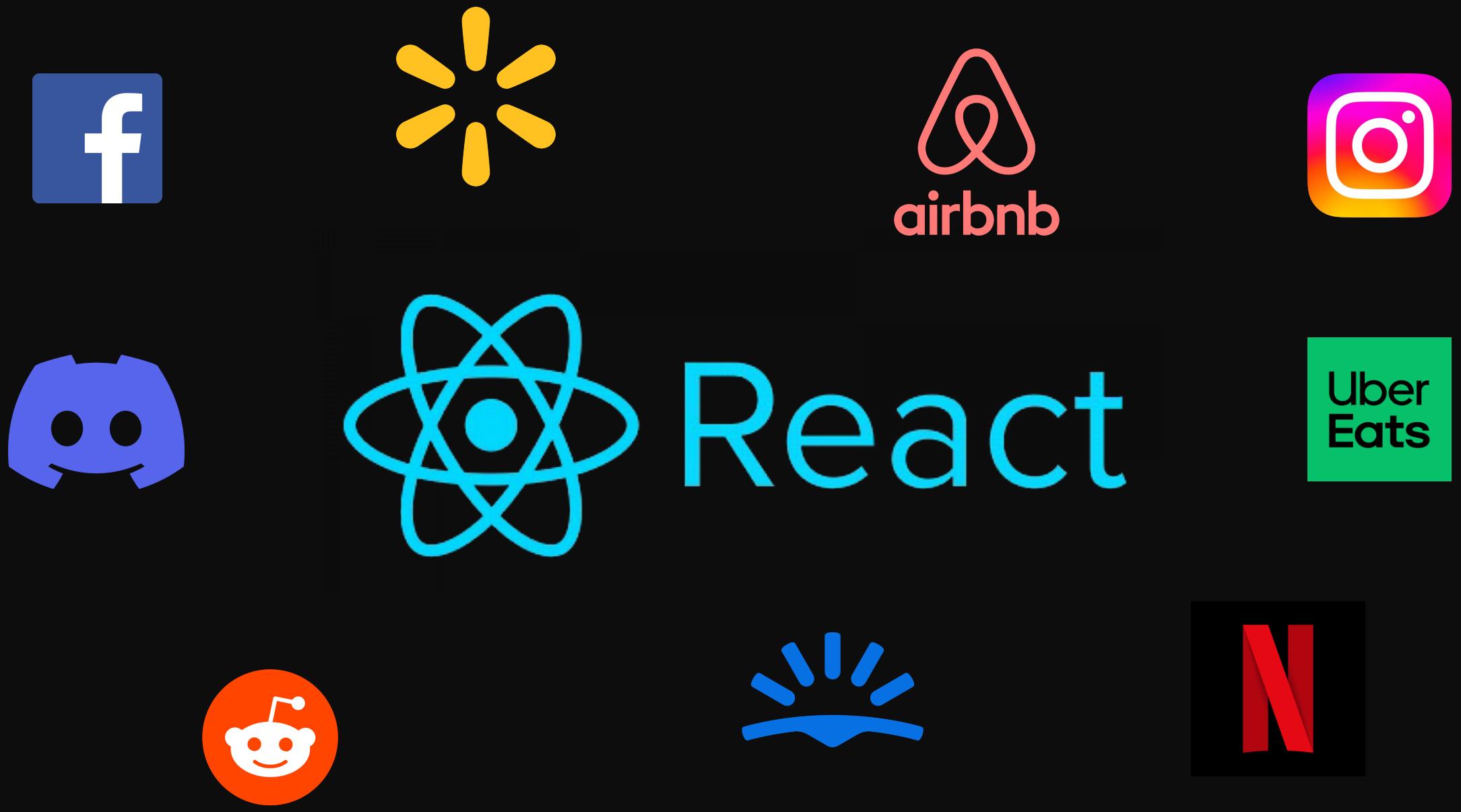


Model-view-controller

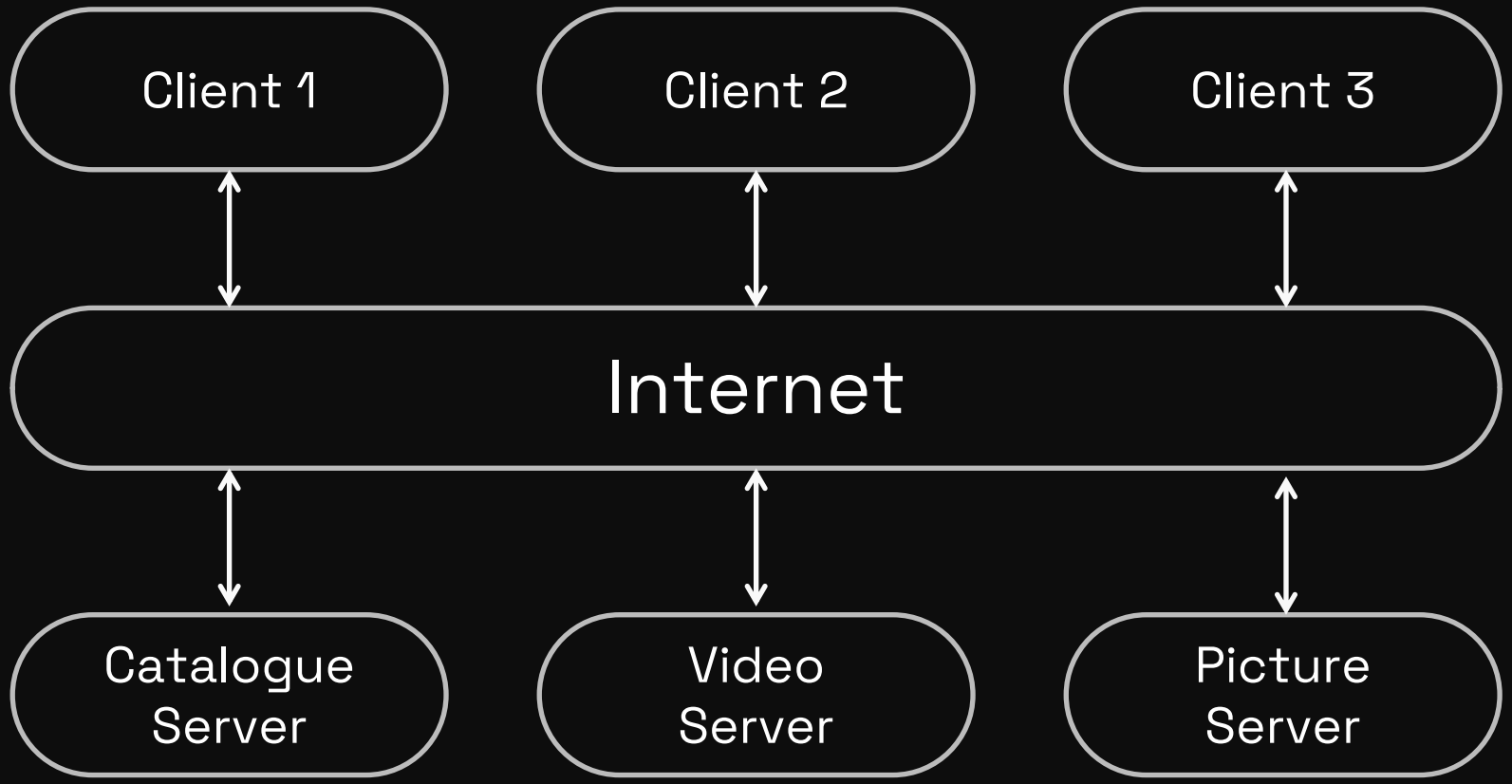


Model-view-viewmodel

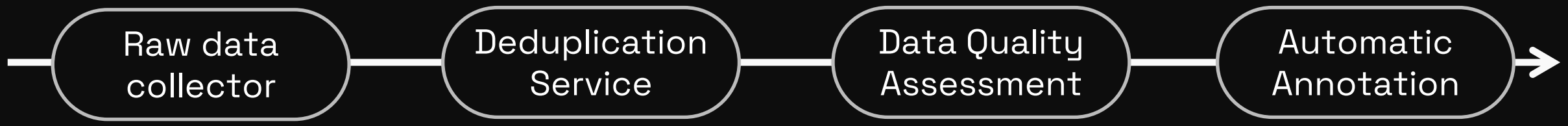



airbnb
React
Uber
Eats

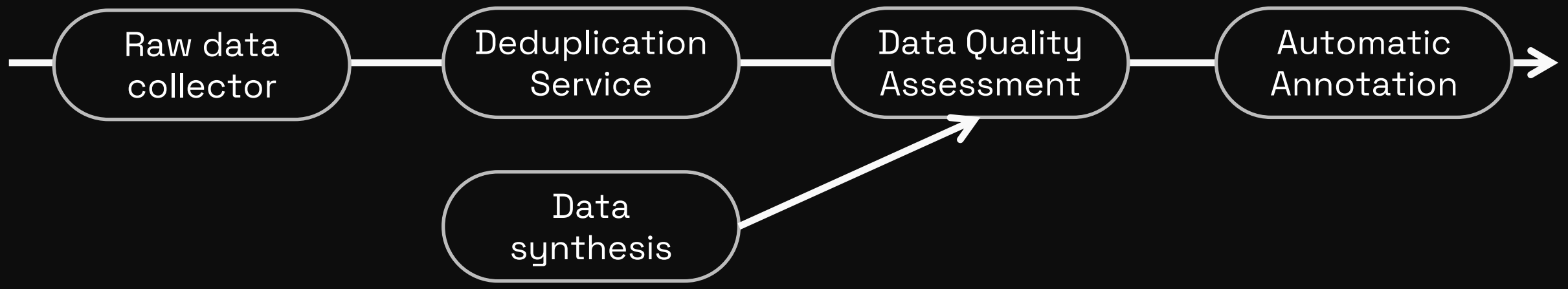
Client-server architecture

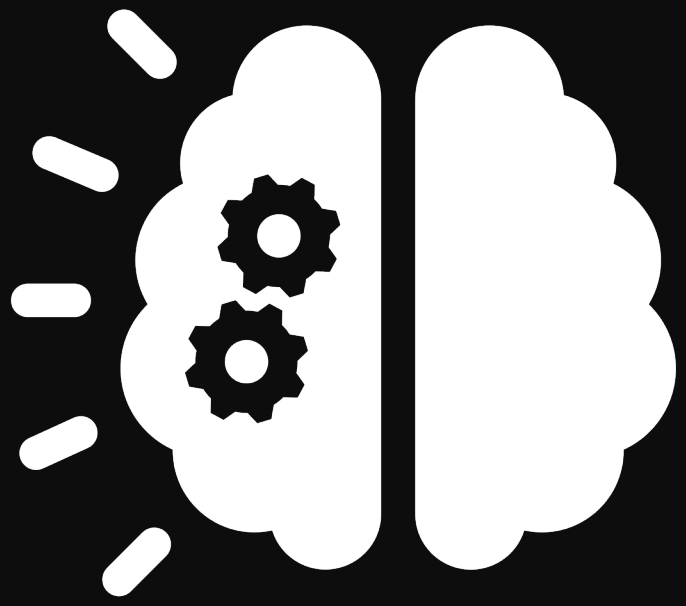


Pipe-filter application



Pipe-filter application

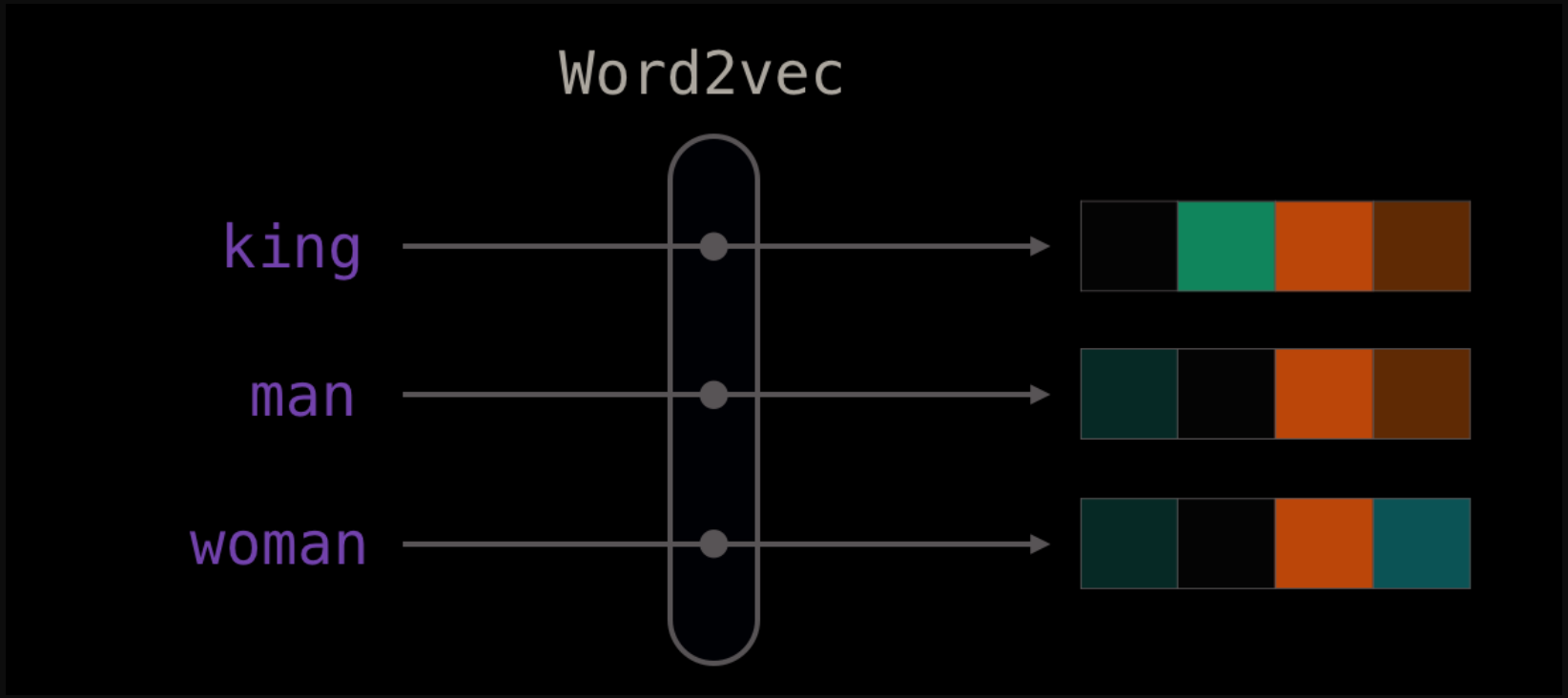


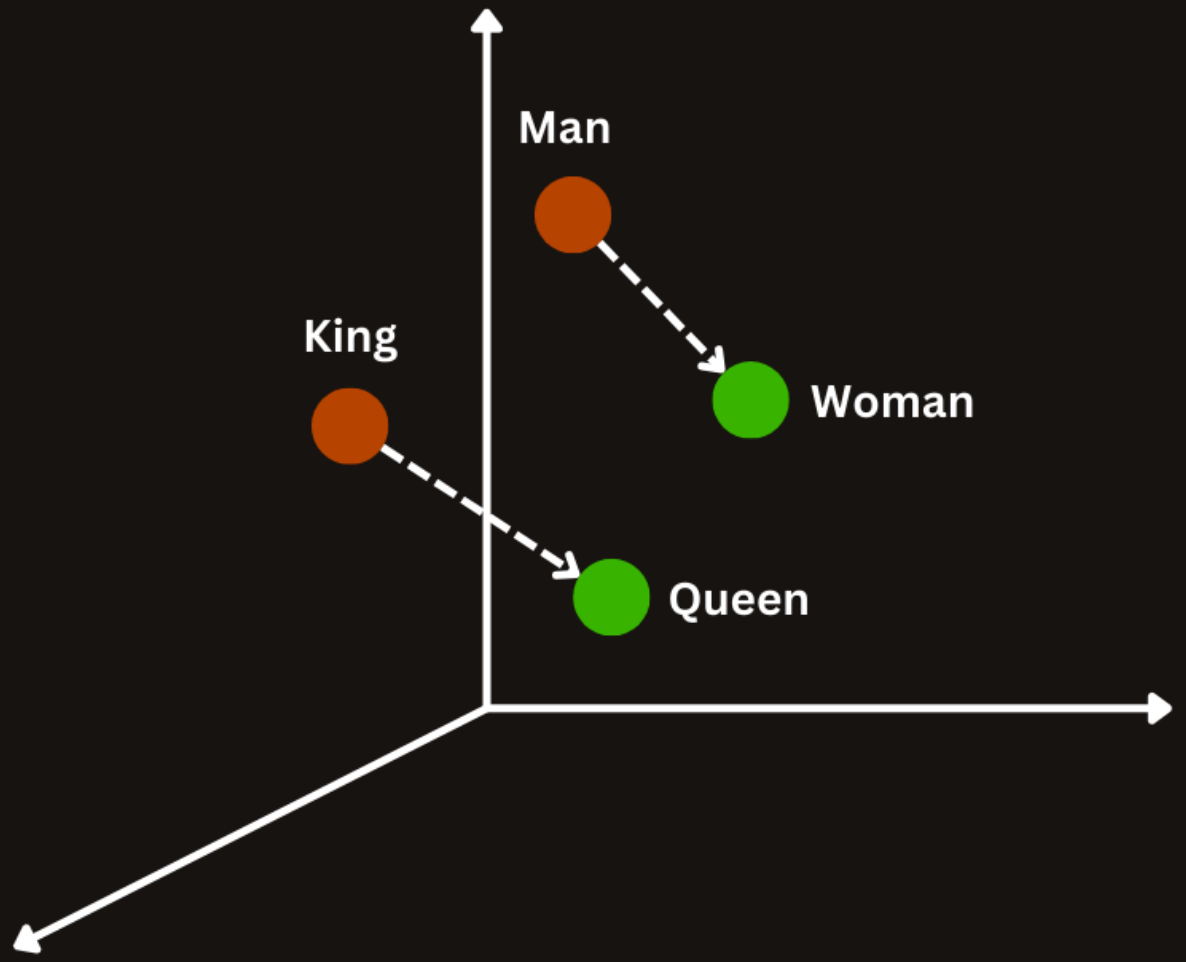


AI SYSTEM PATTERNS

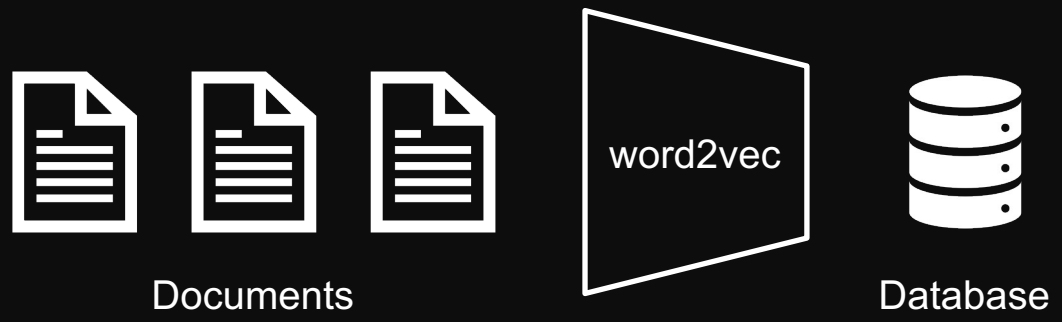
Retrieval-augmented Generation

- Problem: hallucinations and lack of context
- Solution: we insert some information retrieved from documents in the context

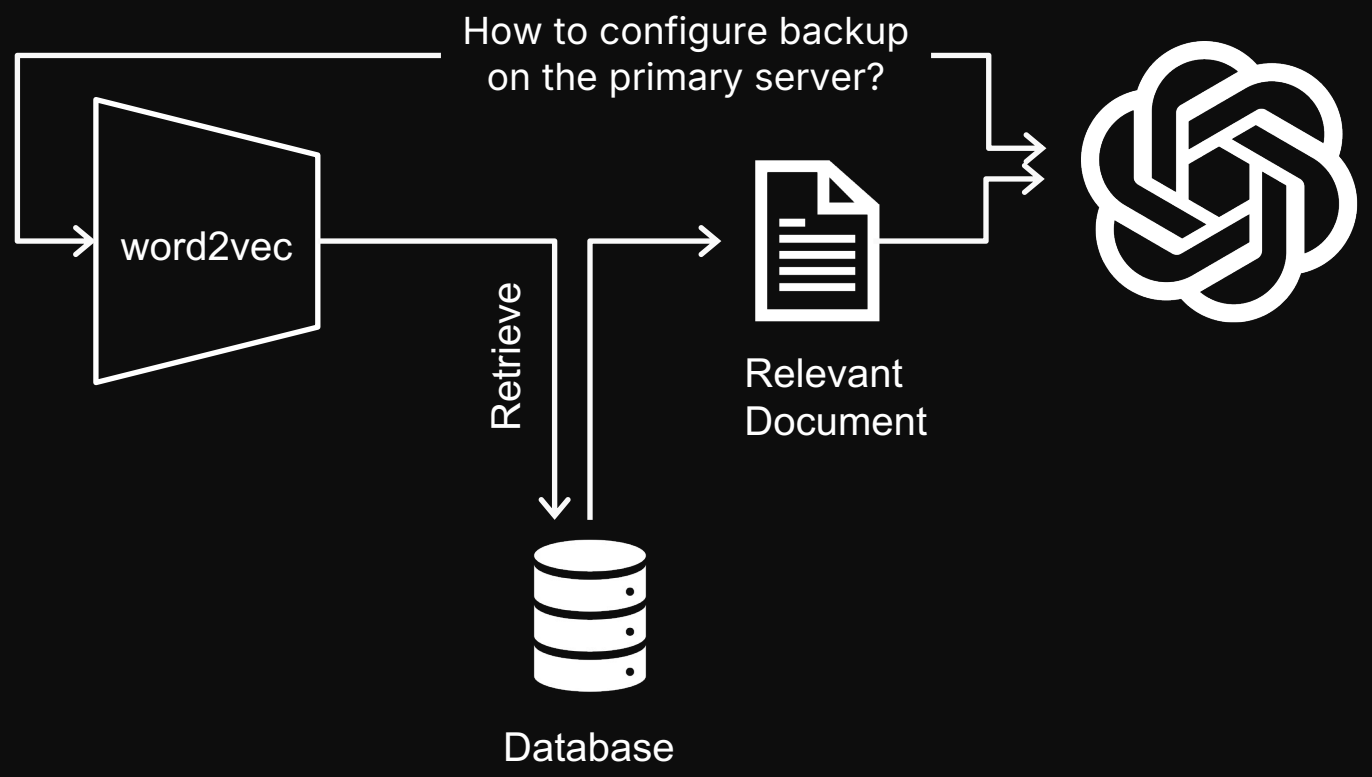

Word2vec
king
man
woman



Word Embeddings


word2vec
Documents
Database

RAG pipeline



AI Agents

- Problem: LLMs cannot execute commands / extract information with existing tools
- Solution: we interact with tools using specific calls

Agentic AI pipeline

TOOL CALL

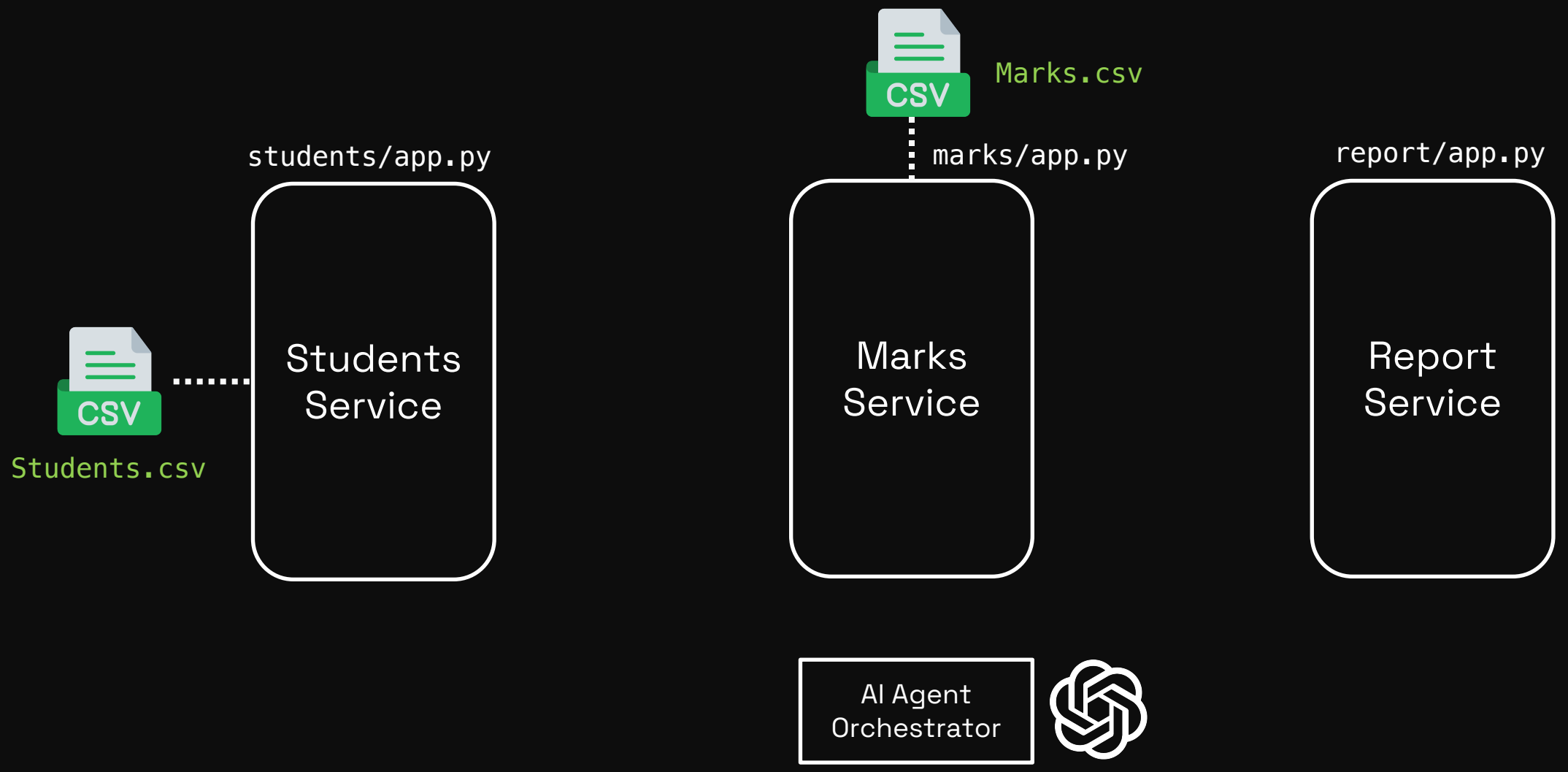
`check_flights()`

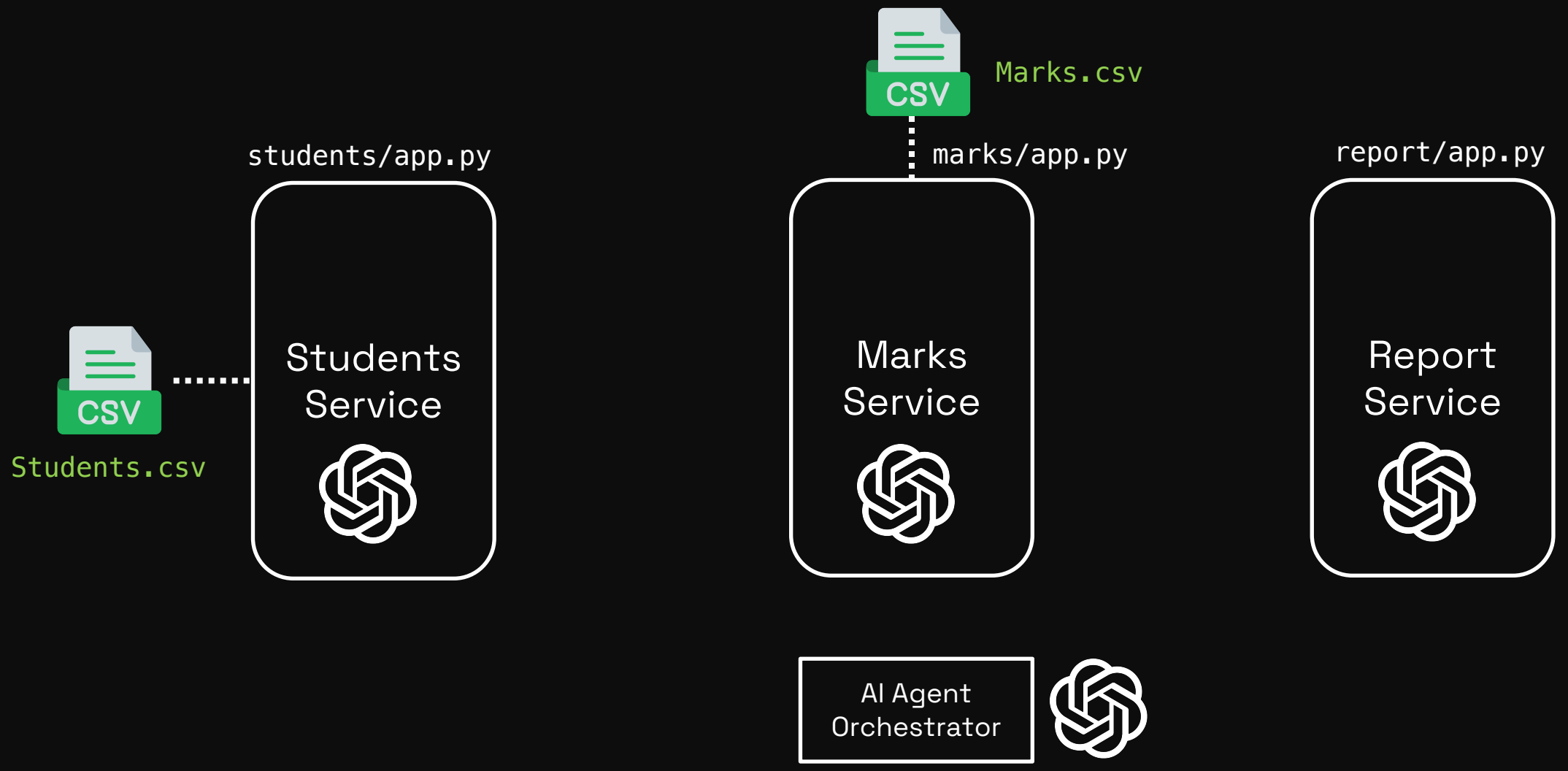
RETURN VALUES

In context!

```
check_flights(
  start_airport: string,
  destination: string,
  date: string
) -> [
  { id: string, price: number, airline: string, time: string }
]

book(id: string) -> confirmation
```


CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
AI Agent Orchestrator


CSV
Marks.csv
students/app.py
marks/app.py
report/app.py
CSV
Students.csv
Students Service
Marks Service
Report Service
AI Agent Orchestrator

Thanks!

References:

Sommerville Chapter 6, Chapter 1
https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith
https://softwareguru.substack.com/p/are-ai-agents-the-new-microservices